Transportation Research Part C 149 (2023) 104055

Contents lists available at ScienceDirect

Transportation Research Part C

journal homepage: www.elsevier.com/locate/trc

Crowdsourced on-demand food delivery: An order batching and assignment algorithm

Michele D. Simoni [a,b,*], Matthias Winkenbach [b]

[a] Division of Transport and Systems Analysis, KTH Royal Institute of Technology, Stockholm, Sweden
[b] Center for Transportation and Logistics, Massachusetts Institute of Technology, Cambridge, MA 02139, USA

ARTICLE INFO

Keywords:
Crowdsourced on-demand food delivery
Batching
Assignment
Simulation
Dynamic routing

ABSTRACT

Since the early 2010s, the meal delivery business went through a veritable revolution due to online food delivery platforms. By allowing customers to quickly order from a wide range of restaurants and outsourcing currently available couriers using their vehicles (crowdsourcing), this typology of service dynamically bridges demand and supply. The main goal of online food delivery platforms consists of matching couriers to meal orders within short time intervals to provide an efficient, reliable, and sustainable service. A way to increase efficiency consists of consolidating orders into batches, such that the same courier can serve several orders in multiple pickup and drop-off routes. Since such an assignment-batching problem becomes computationally prohibitively costly in real-world scenarios characterized by a large number of customer orders as well as uncertain demand and supply, heuristic solution methods come into play. This study proposes an order batching and assignment algorithm that leverages a graph-based approach after decomposing the original problem into more tractable sub-problems employing clustering. The solution is improved by local search moves and re-optimization procedures and integrated with advanced policies to improve solutions over time. An 'Insertion policy' aimed at increasing batch size, and a 'Swap policy' aimed at identifying more efficient assignments, are implemented and compared to a 'Myopic policy' that does not involve any re-optimization over time. An agent-based simulation framework is developed to implement dynamic policies where couriers' operations and movements are realistically reproduced. The performance of the developed solution approach is tested through experiments based on a real-world case study. Results show that the algorithm allows for high-quality solutions in several configurations characterized by different demand and supply patterns (e.g., density levels, couriers availability) and problem sizes.In particular, the two advanced policies investigated considerably improve the solutions.

1. Introduction

E-commerce and business-to-consumer (B2C) services have been growing steadily since the early 2010s, (Vakulenko et al., 2019) across all sectors and industries. In particular, mobile and web-based delivery apps have become increasingly popular in the food delivery industry. Online meal delivery is expected to grow from 130.2 billion dollars in 2022 to 223.7 billion dollars in 2027, according to experts (Coppola, 2022). While the notion of getting freshly prepared meals delivered to the customer's home from a given restaurant has been around for many years, the recent growth of this market is primarily driven by the market entry and

* Corresponding author at: Division of Transport and Systems Analysis, KTH Royal Institute of Technology, Stockholm, Sweden.
E-mail address: micheles@kth.se (M.D. Simoni).

https://doi.org/10.1016/j.trc.2023.104055
Received 15 November 2021; Received in revised form 31 October 2022; Accepted 31 January 2023
Available online 16 February 2023
0968-090X/© 2023 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

dynamic growth of digital On-Demand Food Delivery (ODFD) platforms. Service providers such as GrubHub, Doordash, Uber Eats (US), Swiggy, Zomato (India), Ele.me and Meituan (China), Deliveroo, and Foodora (Europe) have emerged as major industry players with steady growth.

Hirschberg et al. (2016) distinguish two general types of platform-controlled ODFD services, which they refer to as *aggregators* and *new-delivery services*. Aggregators are asset-light ODFD platforms that focus on providing transparent and easy access to the offerings of multiple restaurants through a single interface, streamlining the order process and payment. They do not organize the actual delivery service, which remains restaurant-controlled. New-delivery services, on the other hand, also organize the delivery logistics between restaurants and customers. Within this framework, the supply often consists of crowd-sourced non-professional couriers who are paid based on their service, similar to ride-hailing services such as Uber of Lyft.

ODFD platforms are confronted with several highly complex, large-scale planning and operational challenges that are critical to their commercial viability and success. Specifically, ODFD platforms need to align three main and potentially conflicting objectives. First, to minimize the cost per order incurred by ODFD platforms, they need to employ order batching and assignment (OBA) algorithms that allow for an efficient fulfillment and consolidated delivery of customer orders. The efficient batching of orders and their assignment to the most suitable courier is also an important lever in expanding the effective processing capacity of the ODFD platform. However, the development of efficient OBA algorithms for ODFD services has not been studied intensively in the extant literature (Seghezzi et al., 2021). Second, service reliability, i.e., the ability to guarantee short delivery lead times (typically under an hour) and to reliably deliver within the delivery time window that was promised upon order placement is critical for ensuring customer satisfaction. According to Hirschberg et al. (2016), 60 percent of consumers across all markets cite service reliability as a key determinant of their buying behavior on ODFD platforms. Third, effective OBA is also the key lever to improve the sustainability of ODFD services. More effective order consolidation and more efficient courier deployment can help reduce the total distance driven, thus mitigating the social and environmental externalities of ODFD services related to pollutant emissions, traffic congestion, noise, and safety.

This paper defines and addresses the OBA problem for platform-controlled ODFD services. Specifically, we propose an algorithm that optimizes the ODFD platform's operations by efficiently consolidating ODFD orders into batches and assigning them to available crowd-sourced couriers. Due to its computational cost and complexity, the OBA problem under investigation cannot be solved as a single optimization problem for realistically sized problem instances. Instead, heuristic solution approaches are required that decompose the problem into less complex sub-problems that can be solved and recombined iteratively. Decomposition approaches are an important element of many state-of-the-art heuristic and metaheuristic solutions for complex routing problems. In the existing literature on solving large-scale routing problems, clustering-based decomposition strategies have been shown to perform particularly well (see, e.g., Santini et al., 2021). Most previous works use clustering techniques such as k-means to partition the problem primarily based on the geographical coordinates of the demand nodes to be fulfilled. In this study, we use a hierarchical method to cluster demand on the order level, using a more comprehensive similarity measure. The proposed similarity well captures the spatial proximity of both the pickup and the delivery node(s) and the temporal compatibility between orders (i.e., the potential delays incurred by serving them on a joint delivery trip). In addition, the presented divisive hierarchical approach is computationally efficient and enables the generation of high-quality clusters.

Our proposed OBA algorithm is efficient from a computational perspective as it allows to identify suitable order batches and courier assignments for realistic problem instance sizes (i.e., dozens of orders placed within a few minutes) within seconds, while providing a generally high solution quality (i.e., solutions within a 5 to 10 percent margin of error, depending on the approximation used). Moreover, the proposed algorithm is highly flexible, as it allows us to explicitly control the transformation of the original problem into simpler, more tractable sub-problems using a variety of different approximations (e.g., order clustering, couriers' positions, courier matches to orders). As a result, the modeler/policymaker can evaluate the trade-off between accuracy and efficiency transparently and make decisions accordingly.

Finally, the proposed algorithm addresses the OBA problem's dynamic nature since it can adapt well to varying demand and supply configurations by integrating advanced policies. Due to the high stochasticity of the underlying problem (demand and supply fluctuate over time and space without prior knowledge), the algorithm is designed to allow for different policies involving different re-optimization approaches. Specifically, we study the integration of two advanced OBA policies. The first policy focuses on enhancing solutions by allowing initial batches to incorporate additional orders as they arrive over time (*Insertion*). The primary rationale behind this policy is to increase order consolidation and thus, reduce the overall utilization of couriers. The second policy aims to improve solutions by allowing for the dynamic replacement of initially assigned couriers to batches with other couriers who become available later (*Swap*). The primary goal of this policy is to reduce the total distance traveled to restaurants. The two policies constitute a reasonable and realistic approach to developing high-quality solutions. They are applied separately and compared to a Myopic approach where the OBA is addressed at regular intervals with no changes to the acquired solutions over time.

In order to model these policies and evaluate the performance of the algorithm in realistic problem settings, we developed an agent-based simulation. The proposed framework reproduces courier delivery operations and orders' status individually and dynamically. Therefore, it is possible to adopt a rolling-horizon optimization approach for the OBA problem where the solutions can be partially changed in response to the system's evolution. We demonstrate the practical value and scalability of our proposed method to realistically sized problem instances by applying it to real-world case studies of existing ODFD platforms (specifically, in the city of Bengaluru, India, and an unspecified U.S. metropolitan area).

The remainder of this paper is structured as follows. First, a description of the OBA problem and an overview of the most relevant literature are provided in Sections 2 and 3, respectively. We then describe the methodology of our proposed solution approach in Section 4. In Section 5, we apply our proposed method to a number of realistic problem instances and analyze its performance before deriving a number of managerial insights from our analysis. Finally, we present our conclusion and avenues for future research in Section 6.

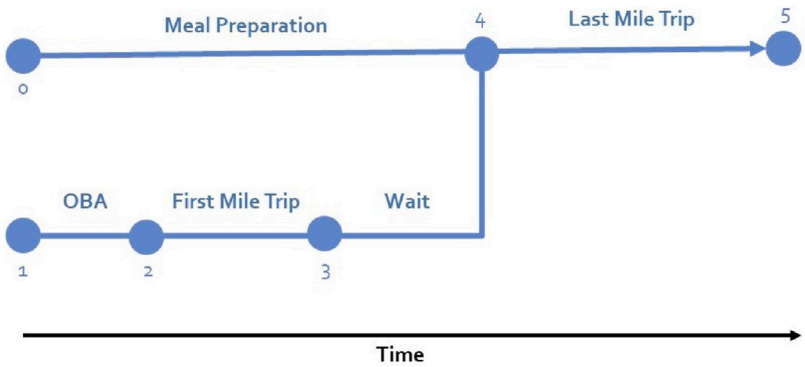


Fig. 1. Food delivery process in platform-controlled ODFD services.

2. Problem setting

On-demand food delivery process. Fig. 1 summarizes the crowdsourced ODFD process. When a customer places an order, the selected restaurant starts the meal preparation (time span 0–4 in Fig. 1). At the time of placement of the order, the ODFD platform provides the customer an estimated delivery time, which we refer to as the Service Level Agreement (SLA) throughout this paper. The promised SLA is based on estimated meal preparation and delivery times, which are typically derived from historical data and real-time conditions of the system. Establishing reliable SLA estimations is in itself a substantial research problem, which, however goes beyond the scope of this paper. For this study, estimated SLAs will therefore be considered as exogenous inputs to the OBA problem. As soon as an order is placed, the ODFD platform starts solving the OBA problem to identify the most efficient batch and courier assignment for the placed order (time span 1–2). Once a crowdsourced courier is identified and notified, he starts traveling to the restaurant, referred to as the *first mile* (time span 2–3). Depending on the OBA problem solution, the courier assigned to the delivery service may or may not arrive before the meal is available for pickup. In the first scenario, there is a *wait time* at the restaurant (time span 3–4). In the latter, the prepared dish waits until the courier arrives and then departs the restaurant (in this case, time point 4 would occur before time point 3). Finally, when the meal preparation is completed, the courier performs the final delivery by traveling to the customer and handing over the order, referred to as the *last mile* (time span 4–5).

Note that each order needs to be picked up and delivered within order-specific pick-up and delivery time windows. These time-window constraints are primarily driven by real-world requirements for food preparation time and freshness on the pick-up side, and the customer SLA, i.e., the promised delivery lead time on the delivery side. For the sake of this study, we consider the time window constraints on both ends as an exogenous input.

Order batching and assignment. The OBA problem for ODFD platforms studied in this paper seeks to identify the most cost-efficient routes for the collection of meals at restaurants and their delivery to customers using a given fleet of crowd-sourced couriers scattered across the city. In the OBA problem, orders can arrive at any time and each order can contain one or more items. Each order needs to be assigned to an available courier within a short amount of time after the order is placed (typically within a few minutes). Assignments need to be made such that the cost per delivery are minimized, while making sure that as many requests as possible can be served. The available couriers are typically independent drivers that can freely log on and off the crowd-sourced delivery system managed by the platform operator at any time. As a result, real-world ODFD platform operation corresponds to a dynamic and stochastic optimization problem, with orders arriving randomly throughout the day and courier availability changing unpredictably as couriers sign in and out of the platform. Due to these characteristics, it is not possible to adopt standard pickup-delivery algorithms since the OBA requires fast and flexible solutions that can be partially improved over time as new information about orders and couriers becomes available.

The OBA problem integrates the following four inter-dependent sub-problems: (i) *order batching*, which aims to consolidate two or more delivery requests in a joint courier trip to increase system efficiency; (ii) *batch assignment*, which aims to match suitable and available couriers to order batches to improve fleet utilization; (iii) *courier routing*, which aims to optimize the pick-ups and drop-offs along the courier's route to maximize travel efficiency; and (iv) *re-optimization*, which aims to improve existing solutions as supply and demand information becomes available. Unlike the conventional pickup and delivery problem with time windows (PDPTW), which typically assume a constant cost per distance and/or time traveled, the courier cost structure in the OBA problem is characterized by (i) a 'first-mile cost' of the distance and/or time traveled to reach the restaurant; (ii) a 'last-mile cost' of the distance and/or time traveled to reach the customer; and (iii) a potential 'waiting cost' penalizing wait times of the courier at the restaurant. Consequently, the OBA problem seeks to minimize the sum of courier travel and waiting cost. In addition, a potential 'penalty' costs for unassigned orders is considered. These are orders whose assignment is postponed to the next re-optimization step because of a temporary lack of supply to satisfy current demand. In the solution approach presented in this paper, we define this penalty as the sum of a fixed cost and a variable cost component depending on the urgency of the order itself (see Sections 4.2.2 and 4.2.3 for further details). Based on this penalty formulation, orders with shorter delivery lead times will be prioritized in the assignment.

3. Literature review

3.1. Pickup and delivery problems

The OBA problem for crowd-sourced delivery services is similar to some well-known optimization problems that have been discussed in the literature. The pickup and delivery problem (PDP) and its variants are the archetypal problem at the heart of a wide range of currently addressed ride-hailing services for both passenger and goods mobility (cf., Savelsbergh and Sol, 1995). In its simplest form, a single vehicle departing from a depot must collect several orders and deliver them in the shortest time possible before returning to the depot. Over the last thirty years, several extensions to the classical PDP have been proposed to account for more complex and realistic settings and constraints, such as the availability of multiple vehicles, the presence of time-windows, and the cost associated with delays and wait times. Among others, Cordeau et al. (2008), Berbeglia et al. (2010), Lahyani et al. (2015), and Battarra et al. (2014) provide comprehensive reviews of the extant literature on the PDP and its variants. In this section, we focus on reviewing the most relevant papers related to the problem setting presented in Section 2, as well as studies that relevant in the context of our proposed solution technique.

A highly relevant variant of the classical PDP consists of the VRPPDTW. Here, similarly to the OBA problem, a fleet of vehicles must collect and deliver goods characterized by their own specific origins and destinations (often referred to as 'one-to-one' problems). The main difference between the VRPPDTW and the OBA problem consists in the VRPPDTW given fleet of courier vehicles departing from (and returning to) one or more depots, while the OBA problem assumes a fleet of crowd-sourced, randomly located couriers that are not tied to any depot. Being an extension to the classical VRP, the VRPPDTW is known to be NP-hard. For this reason, only relatively small instances can be solved exactly. Dumas et al. (1991) solve instances of the VRPPDTW with up to 55 customers by means of an exact-column generation algorithm. Tackling the same problem by means of branch and cut algorithms based on the generation of several infeasible path inequalities, Ropke and Pisinger (2006), Ropke and Cordeau (2009) optimally solve problem instances with up to 175 customers. Baldacci et al. (2011) also propose an exact approach based on column generation with high computational efficiency. Other promising heuristics include the combination of linear relaxation and set-partitioning (see, e.g., Xu et al., 2003); evolutionary metaheuristics (see, e.g., Pankratz, 2005); large neighborhood search (see, e.g., Bent and Van Hentenryck (2006), Ropke and Pisinger (2006)); and dynamic programming applied to state-space–time networks (see, e.g., Mahmoudi and Zhou, 2016). Sun et al. (2018) present a variant of the VRPPDTW in which the operator can selectively accept requests in order to maximize profit.

Another relevant variant of the PDP consists of the DARP. Here, multiple vehicles need to serve several customers who specify pickup and delivery requests between origins and destinations (Cordeau and Laporte, 2007). Since the DARP was originally focused on the transportation of people, customer inconvenience due to wait times and route duration is explicitly modeled through a penalty cost. In the OBA problem, courier wait times at restaurants and freshness-related constraints are explicitly included in the objective function, while the overall route duration of a delivery trip does not affect the appeal of potential order batches (as long as deliveries are made according to the SLA, i.e., within the promised delivery lead time). In the past twenty years, efficient heuristics have been developed to solve this problem by means of tabu search (Cordeau and Laporte, 2003), variable neighborhood search (Parragh et al., 2010), deterministic annealing (Braekers et al., 2014), and dynamic programming (Ritzinger et al., 2016). This variant of the PDPs has gained significant attention in recent years due to the growth of ride-hailing services, such as Uber and Lyft. Some relevant applications can also be found in the context of shared autonomous mobility services (Bongiovanni et al., 2019; Liang et al., 2020). However, in most of the above-mentioned studies, vehicles depart from and return to one or more depots, unlike the OBA problem studied in this paper.

Both VRPPDTW-related contributions reviewed above focus on static problem variants where demand (i.e., the set of transportation requests) and supply (i.e., the available courier fleet) are fully deterministic and known a priori. None of the reviewed papers employs any specific approach for the optimal use of gradually disclosed demand and supply information, which would characterize dynamic variants of the PDP and its extensions. Over the past decade, the family of dynamic and stochastic vehicle routing problems (VRPs) and PDPs has received increasing attention in the academic literature, with a variety of methodological advances and applications ranging from ride-sharing services to the integration of real-time traffic information (see, e.g., Braekers et al., 2016, for a review and classification). The interested reader may refer to Ho et al. (2018) for a comprehensive review of dynamic VRPPDs and DARPs. In the specific context of ODFD, policies can be typically developed to overcome the limitations of myopic approaches that solve the dynamic problem by breaking it down into several independent static sub-problems. For example, Reyes et al. (2018) propose so-called 'commitment strategies' to maintain flexibility in the order assignment while accounting for order urgency. Steever et al. (2019) propose a 'pro-active' heuristic approach to anticipate future demand needs. Yildiz and Savelsbergh (2019) develop an exact solution approach for scenarios with perfect information about order arrivals to investigate capacity planning and demand management strategies.

A popular solution approach to such dynamic and stochastic problems is known as the *rolling horizon* approach. For example, Arslan et al. (2019) propose a heuristic and an exact algorithm to solve a crowdsourced parcel delivery service formulated following a rolling-horizon approach. A rolling-horizon approach breaks down the overall dynamic and stochastic optimization problem into a sequence of static and deterministic sub-problems according to regular time intervals, also referred to as *re-optimization steps* in the following. By periodically solving the static and deterministic version of the problem at every re-optimization step and incorporating new information about the (real or simulated) system obtained since the last re-optimization, the problem becomes tractable (cf., Agatz et al., 2011). The sub-problems solved at every re-optimization step can exhibit various degrees of inter-dependency. In the simplest case, the *myopic rolling horizon* approach, each re-optimization step is considered separately from previous and

future ones (although decisions made for one period clearly affect the input of the following optimization problems). In the context of this study, the OBA problem would hence be reduced to identifying the best possible solution for the current demand and supply configuration at the current re-optimization step, regardless of decisions made in previous steps, or decisions to be revised later. More *advanced rolling horizon* approaches exploit inter-dependencies of decisions made in consecutive re-optimization steps to obtain more favorable solutions. For instance, in the context of an OBA problem, allowing for the re-optimization of order batches formed in previous re-optimization steps, postponing specific order batch assignment decision to later re-optimization steps, anticipating future orders, or allowing for flexible adjustments to existing courier routes, can yield solutions with significantly better commercial and operational performance. However, this additional complexity comes at significantly higher computational cost (Mitrović-Minić and Laporte, 2004; Stiglic et al., 2016; Reyes et al., 2018; Steever et al., 2019). In this study, we develop and investigate an OBA algorithm that is suitable for both the myopic approach and more advanced policies.

3.2. Shared mobility systems and crowd-shipping

Another related research stream consists of studies on shared-mobility services, crowd-shipping services, and dynamic ride-sourcing for goods delivery, which has flourished in recent years due to the rise of ride-hailing businesses and ODFD platforms. Similarly to shared mobility services for passengers, transportation service is outsourced to non-professionals whose availability is uncertain and variable (couriers can freely log in and out of the system). One distinguishing feature of the studied problem consists of the (potential) couriers' wait at restaurants which is considered in the couriers' compensation and the related optimization problem. Furthermore, while the optimization problem for ride-hailing operators typically aims to minimize customers' waits, the ODFD operator seeks to reduce costs while complying with delivery time commitments (SLA constraint) in the considered OBA problem. Mourad et al. (2019) provide an overview of the corresponding optimization problems and proposed solution approaches that have been proposed in the shared mobility space. Similarly, Le et al. (2019) provide a comprehensive review of how crowd-shipping services are discussed in practice and academic research from a supply, demand, and operational perspective. More recently, Le et al. (2021) focus on the pricing and driver compensation perspective of crowd-shipping services by combining driver matching and routing models in an integrated analysis framework. Most recently, a number of authors have proposed model formulations and solution methods for the optimization of crowd-shipping services incorporating a variety of real-world complexities that are also relevant to the ODFD problem, such as time windows, transshipment operations, and alternative delivery options (see, e.g., Macrina et al., 2020; Vincent et al., 2022). Wang and Yang (2019) offer a broader analysis of different shared-mobility and crowd-shipping related problems (operational to business-related ones) together with a comprehensive framework of related stakeholders, agents, and attributes. Among recent works, the most relevant studies are summarized below. Alonso-Mora et al. (2017) propose a graph-based approach to identify potential matches between taxis and people with a real-world application to New York City, USA. Along the same lines, Simonetto et al. (2019) combine linear programming and context mapping to tackle the same problem with a focus on multiple capacity providers (here, taxi companies). Qian et al. (2017) investigate the same problem and analyze incentives to increase the observed levels of ride-sharing. It is worth mentioning that, just like in our study, these papers focus on the myopic version of the problem, where the transportation requests and their assignments to carriers are defined and optimized over a certain interval of time, with subsequent time intervals being assumed to be independent. The obtained solutions may therefore not correspond to the global optimum across an extended period (e.g., a business day). Other studies consider the dynamic nature of the problem more explicitly. For instance, Ma et al. (2019) combine a routing approach with a repositioning strategy integrated with transit service to optimize overall mobility, while Hyland and Mahmassani (2018) investigate alternative assignment policies using an agent-based model. Bertsimas et al. (2019) employ a re-optimization approach based on a backbone solution that could be iteratively updated with dynamically incoming information. Hess et al. (2021) point out that accurate, location-specific near-term predictions of delivery requests are a critical lever to improve the operational efficiency of ODFD platforms by anticipating the time and location of demand occurrences. The authors compare a number of established and newly proposed forecasting methods to guide future research in this direction. Finally, a few authors address the challenge of reaching profitability for mixed fleets of regular drivers and crowdsourced riders (Masoud and Jayakrishnan, 2017) and for systems adopting flexible pricing (Chen et al., 2020).

Methodologically, various studies whose solution methods are applicable to our proposed algorithm have been established. First, since the possible batching solutions in our research are expressed using a graph-based approach, related studies in the field of shared-mobility analysis are presented. Santi et al. (2014) and Alonso-Mora et al. (2017) introduce the concept of 'shareability networks' as a novel approach to solving large vehicle-pooling problems. This approach translates possible combinations of shared trips into a graphical representation (tri-partite graph) that can be optimally solved through existing assignment algorithms. Santi et al. (2014) employ this approach to show how a considerable number of taxi trips (up to 80 percent) in New York City, USA could be shared. However, the number of candidate taxis evaluated for each shared trip is limited. Alonso-Mora et al. (2017) overcome this limitation by extending the algorithm with pruning and insertion heuristics that allow for more computationally efficient solutions. Additionally, imposing additional limitations on customers' maximum wait times speeds up computations. Similar approaches have been adopted by Vazifeh et al. (2018) to study the optimal fleet design problem and Rasulkhani and Chow (2019) to analyze alternative pricing strategies. To the best of our knowledge, none of these studies explicitly adopts a dynamic optimization perspective that seeks to improve the solution over time. In this work, we propose an OBA algorithm that employs a spatial partitioning approach to break down the problem into several independent sub-problems to address computational challenges. Similar clustering approaches have been identified in the literature. Among others, Häme and Hakula (2015) and Mahmoudi et al. (2019) present approaches to cluster orders/rides based on their origins and destinations, following the seminal work of Bodin and Sexton (1986) and Desrosiers et al. (1986).

4. Methodology

In this study, we propose an OBA algorithm following a rolling horizon approach for ODFD platforms. This is a popular solution technique to such dynamic and stochastic problems. It decomposes the overall optimization problem into a series of sub-problems that are solved in regular time intervals (Δh), which are referred to as *re-optimization steps*. At each re-optimization step, new information from the system (e.g., new orders, courier logins/logouts, operations that) is incorporated and considered in the sub-problem depending on the adopted policy. We consider a policy, any strategy involving the re-optimization of batches and assignments in response to newly acquired system information. The system, which can correspond to either the real-world or a simulation, updates at a regular time step Δt (typically, much shorter than the optimization step). As discussed previously, real-world ODFD systems require solution approaches that yield very short solution times at each iteration of the rolling horizon procedure due to their high order arrival rates and short re-optimization intervals. Regardless of the policy adopted within the rolling horizon framework, the derivation of good-quality partial solutions at each re-optimization step is fundamental for achieving overall efficient ODFD operations. Therefore, a crucial goal of our proposed OBA algorithm is to efficiently solve the static and deterministic sub-problem occurring at every re-optimization step. In Section 4.1, we briefly summarize the general solution framework we employ in this paper, before presenting the various steps of our proposed OBA algorithm in detail in Section 4.2.

4.1. Simulation-optimization framework

To capture the dynamic and stochastic nature of ODFD systems, we need to solve a potentially interdependent sequence of sub-problems as the system state evolves and new information (i.e., randomly incoming orders, couriers' logins and logouts) is revealed over time. In this study, we assume the ODFD operator has no information on future demand and supply. Therefore, the goal of the sequential stochastic problem to be solved is to find an optimal (or well-performing) policy, accounting for the current system state and its evolution over the past steps. In this study, we focus on approaches that involve partial re-optimization of OBA solutions based on the system evolution (and corresponding newly acquired information) between optimization epochs.

The corresponding re-optimization problem can be defined as follows. The set of orders over the entire time horizon T is denoted as O. Each order o_i is defined by a pick-up point (i.e., the restaurant), p_i, and drop-off point (i.e., the customer), q_i. These points are characterized by the time constraints t_r and t_q, respectively, to denote the meal preparation time and the promised delivery time. The set of couriers that become available anytime throughout the entire time horizon is denoted as D. To cater to both myopic and advanced rolling horizon approaches, our proposed algorithm allows for different levels of interdependence between consecutive re-optimization steps. At the optimization epoch h_i, two subsets $O*$ and $D*$ are considered in the modified version of the previously described PDPTW problem where decisions x_h involving routes need to be taken based on the system conditions S_h. The subset $O*$ includes all orders placed during the latest re-optimization interval (Δh) (which are still unassigned) and a subset of previously assigned orders that fulfill conditions given by the chosen OBA policy. For example, if the *Insertion* policy is adopted, single order batches from previous epochs will be considered with newly arrived ones. The same concept applies to $D*$ where the OBA policy indicates which couriers are considered for assignment. Regardless of the policy selected, only couriers who are idle and ready to begin their delivery trip are considered. For example, previously assigned couriers could be considered in addition to all those available couriers in the system at time $t = h_i$. Essentially, each OBA policy determines the orders and couriers considered in the optimization problem, together with a set of constraints. This process can be formally described by

$$\min_{x_h}\{C(x_h, S_h(D*, O*))\}. \tag{1}$$

The re-optimization procedure needs to be embedded in a simulation that reproduces the evolution of the system S in terms of courier availability and incoming orders. The simulation reveals new information about orders that have been placed and couriers that are available, which needs to be taken into account by the re-optimization procedure. The solutions of the re-optimization, in turn, are fed back to the simulation in the form of courier tasks. Fig. 2 shows a high-level overview of this simulation-optimization framework and illustrates how the simulation model connects to the re-optimization model (i.e., our OBA algorithm).

In this study, we adopt an agent-based simulation that reproduces arriving orders, couriers, customers, and restaurants involved in the system as *agents* capable of performing different tasks. Agents' tasks are updated dynamically throughout the simulation according to regular time intervals (Δt), typically of a few seconds (fixed-increment time progression). The list of courier tasks include 'traveling' (to a restaurant or a customer), 'waiting' at restaurant, and 'picking up' and 'dropping-off' orders. The agents' states are updated according to their performance of tasks. When traveling, couriers' positions on the road network are updated based on their speed. A courier is considered idle (without tasks) when logging in the system and when completing a delivery. Similarly, an order is considered unassigned, when it appears in the system. For every re-optimization interval (Δh, typically one or two orders of magnitude larger than the simulation interval Δt), information from the system are used as input for the OBA algorithm and results from the optimization process fed back as new or changed tasks. The simulation is run for a given fixed amount time, and the process repeats until termination ($t > T$), when the output data are stored for post-simulation analyses.

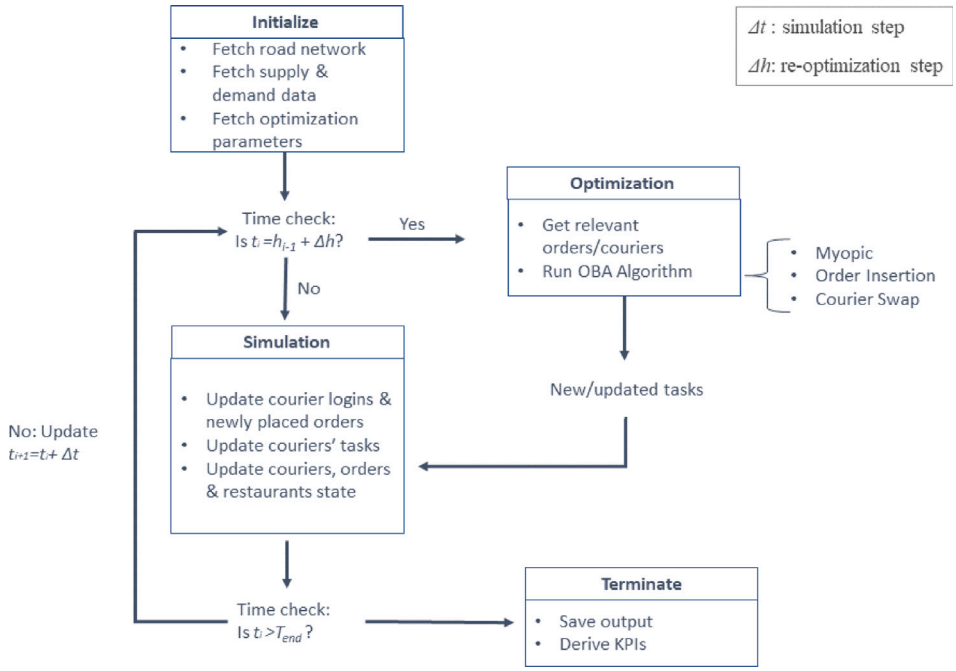


Fig. 2. Simulation-optimization framework.

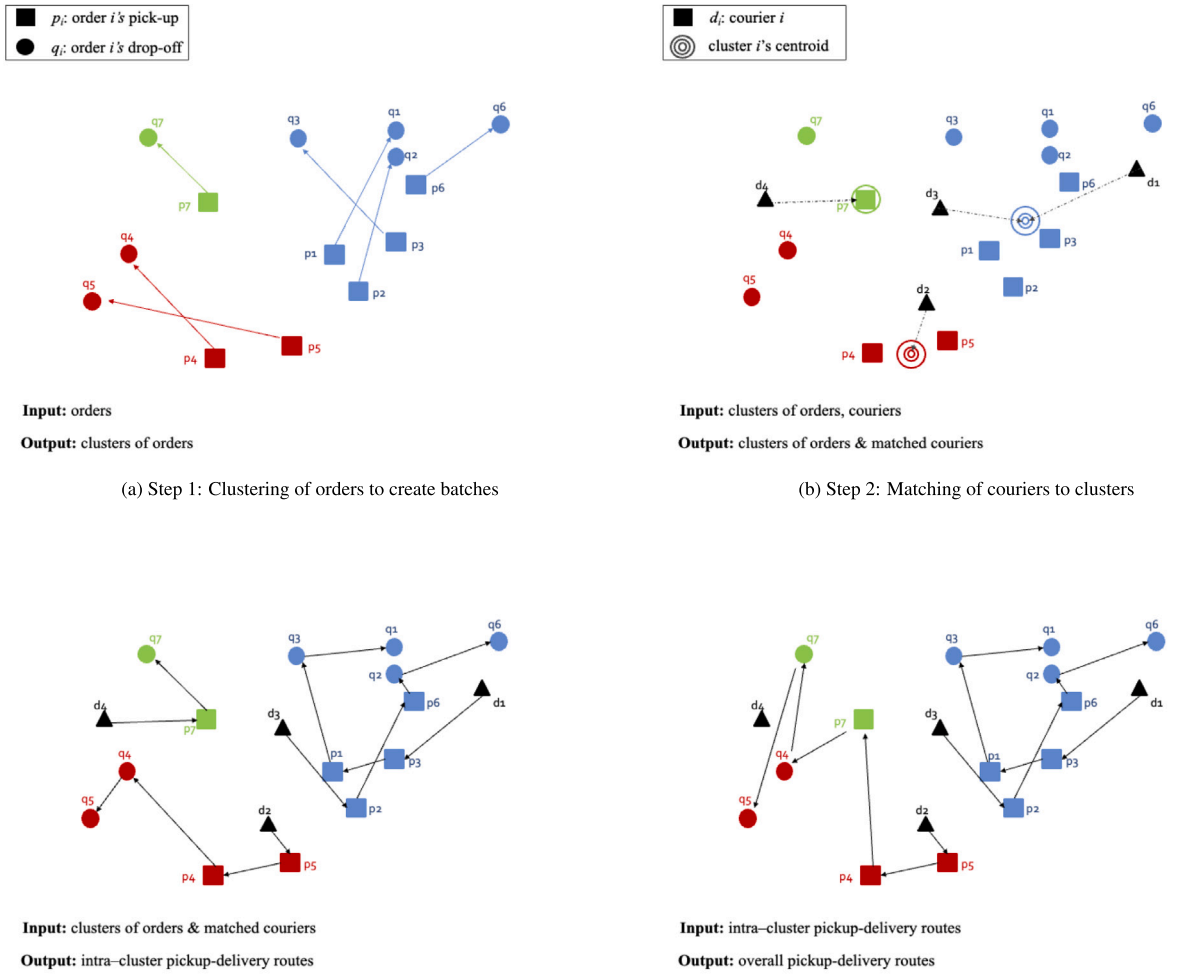
4.2. Order batching and assignment algorithm

To increase computational performance, the proposed OBA algorithm involves dividing the problem into smaller, more manageable sub-problems that can be tackled directly. The sub-problems' solutions are then merged to provide a solution to the original problem. This approach is similar to a 'divide-and-conquer' paradigm that divides the main problem into sub-problems, solves them recursively, and finally combines the solutions to solve the original problem. Another prominent feature of the proposed algorithm consists of the supporting graph structure that encodes alternative routes and allows the adoption of traditional optimization techniques. Depending on the adopted policy, such a graph structure is differently updated throughout different optimization steps. In its most basic version, referred to as the *Myopic* policy, the optimization problem only focuses on orders arrived during the latest optimization interval and identifies OBA solutions given the available couriers. In advanced policies, such as *Insertion*, and *Swap*, OBA results from previous optimization intervals are partially considered too.

Given the adopted 'divide-and-conquer' approach, multiple sequential steps are carefully designed to guarantee a high performance of the algorithm. The first four main steps, which are common to every policy, are illustrated in Fig. 3. The first step, which we discuss in Section 4.2.1, consists in the clustering of orders that are more likely to yield cost savings when batched together. The second step, discussed in Section 4.2.2, consists of matching available couriers to the order batches generated in the previous step, based on their approximate time and distance from these clusters. At this stage, no actual PDP route nor match between batches and couriers has been defined yet. In the third step, discussed in Section 4.2.3, an actual multi-vehicle pickup-and-delivery problem with time windows (mPDP-TW) is solved for each batch of orders and its pre-assigned couriers. In the fourth, discussed in Section 4.2.4, different local search (LS) and combination procedures are implemented in order to improve the obtained solutions. Such a procedure explores potential inter-cluster batching and assignment solutions and solves the mPDP-TW globally. In the fifth step (Section 4.2.5), which is implemented only in the advanced policies (*Insertion* and *Swap*), the obtained solutions are integrated with the solutions from previous optimization epochs according to the selected policy. This process is achieved by partially transferring the developed graph structure over consecutive optimization intervals. Table A.2 summarizes the notation used in the following sections.

4.2.1. Step I: Clustering

The main rationale behind this initial step is to make the problem more tractable by reducing it to several independent sub-problems that only contain orders that are likely to be served by the same courier in the optimal solution to the problem. The grouping of orders into batches relies on clustering analysis, which represents a family of unsupervised machine learning techniques. Specifically, we adopt and modify the Divisive Analysis (DIANA) clustering method (Rousseeuw and Kaufman, 1990), a hierarchical algorithm in which a single cluster is recursively split it in smaller clusters (Rokach and Maimon, 2005). DIANA presents several advantages over other clustering techniques for the studied problem. It requires little input and, therefore, little effort to estimate the parameters. For instance, unlike k-means clustering, it is neither constrained by a predefined target number of clusters nor by neighbor and core sizes (unlike DB-SCAN and H-DBSCAN). Additionally, hierarchical algorithms are less sensitive to outliers than



(a) Step 1: Clustering of orders to create batches

(b) Step 2: Matching of couriers to clusters

(c) Step 3: Solving a mPDP-TW within each cluster

(d) Step 4: LS and re-optimization to improve solution

Fig. 3. Core phases of the proposed OBA algorithm (same for any policy).

other methods (Ding and He, 2002). Finally, it is simple to manipulate. In the proposed extension, its recursive splitting mechanism is stopped anytime a branch reaches a 'small enough' size (with significant computational savings when large problems are addressed). A preliminary comparison with other well-known methods, such as DBSCAN and H-DBSCAN revealed better performance in terms of 'Silhouette score' (which measures the quality of any clustering approach).

Like most other clustering methods, DIANA employs a distance or 'similarity measure' to reflect the strength of the relationship between any pair of data points (here, between any pair of orders). To this end, we derive a similarity measure for each pair of orders, which is based on the cost increase/decrease obtained by serving them on a joint route versus the cost of serving them each individually. To calculate this cost differential, we consider only the journey from the restaurant to the corresponding customer (last-mile cost). We thus abstract from the specific couriers used to serve these orders and their respective distances to the orders. Since the final cost needs to be ultimately related to the time dimension, we consider potential delays resulting from batching orders by means of an arbitrarily high delay penalty. If any of the two orders arrives late, a large penalty is added to the cost, lowering its similarity score. This method helps identify undesirable matches for clustering. Following this general idea, a similarity matrix of all pairs of orders can be established according to Algorithm 1.

Once the similarity between orders is calculated, a modified version of DIANA clustering is performed. Initially, all data is gathered in the same cluster. Just like in the original algorithm, clusters can be divided until each cluster contains only a single observation. At each stage, the cluster with the largest diameter is selected (the diameter of a cluster is the largest dissimilarity between any two of its observations) and split. In our modified version, if the dimension of a newly created cluster is below a pre-defined threshold size, such a cluster can be ignored in the following algorithm iterations. This artifact is introduced to reduce the computations for large instances (the splitting procedure does not need to continue once the threshold is reached) and to guarantee

Algorithm 1: Similarity matrix computation

Inputs: pair of orders (o_i, o_j) with corresponding last-mile costs $(\hat{k}_i, \hat{k}_j)$;
for *all potential sequences of pick-up and drop-off points* $\in (o_i, o_j)$ **do**
  Compute the cost k of serving this sequence ;
  Update the minimal cost $k*$ if necessary;
  Add $s_{ij} = \frac{k*}{(\hat{k}_i + \hat{k}_j)}$ to S
end
Outputs: Similarity matrix S ;

a certain degree of homogeneity in the resulting clusters. The interested reader is referred to Rousseeuw and Kaufman (1990) for further details on the original algorithmic steps of DIANA, whereas the resulting modified divisive clustering algorithm is summarized in Algorithm 2. The optimal number of clusters can be selected based on alternative measures for the evaluation of the 'goodness of the clustering', such as the Silhouette score, the Davies–Bouldin score, and Calinski and Harabasz score (Liu et al., 2010). In this study, we adopt the Silhouette score (or silhouette coefficient), which measures how similar one point is to its own cluster compared to other clusters and can be calculated for each point in the data set as

$$\phi_i = (b(i) - a(i)) / \max(a(i), b(i)), \tag{2}$$

where $a(i)$ corresponds to the mean distance between i and all other data points in the same cluster, $b(i)$ corresponds to the smallest distance between i and all points in any other cluster, of which i is not a member. The score ranges between -1 and 1. Higher mean values of points $\bar{\phi}$ in a cluster indicate tightly grouped data in their cluster assignment (therefore, a well-defined cluster).

For our purposes, the 'optimal' number of clusters is chosen according to different criteria. The most important consists of the maximum value of the average Silhouette value $\bar{\phi}_i$ for all identified configurations of clusters i. In addition, additional criteria on the maximum cluster size and number can be imposed in order to control the computational efforts required for solving the following steps of the algorithm. For example, a maximum cluster size can be imposed to control the computational effort of solving the corresponding mPDP-TWs solved in Step III. Limiting the number of clusters can help keeping the effort associated to the LS and re-optimization improvement heuristic in Step IV of the algorithm manageable.

Algorithm 2: Clustering algorithm

Inputs: Set of orders O, Similarity Matrix S, maximum cluster size T, minimum cluster size T';
Let χ_0={O} be the original cluster corresponding to the initial configuration of clusters X_i=X_0
while *all clusters* $\|\chi_i\| \in X_i > T'$ **do**
  Identify cluster χ_d with largest diameter (highest similarity value);
  Identify the splinter element z as the one with the highest average similarity within the cluster $\chi_d \in X_i$;
  z=max($\bar{s}_{z_i}$) $\forall i \in \chi_d$;
  Initiate splinter cluster χ_s;
  Reassign orders between the original cluster and the splinter cluster;
  for *every order* $o \in \chi_d \setminus z$ **do**
    if $\bar{s}_{oj}$-$s_{oz} < 0$ **then**
      Add order o to the splinter cluster χ_s
    end
  end
  Calculate the average Silhouette Score $\bar{\phi}_i$ for the current cluster configuration X_i;
  Store cluster configuration X_i and corresponding Silhouette Value $\bar{\phi}_i$;
end
Identify the maximum Silhouette Value $\bar{\phi}* = \max(\bar{\phi}_i)$ and the corresponding "optimal" cluster configuration $X* = X_i$;
while *all clusters* $\|\chi_i\| \in X* < T$ **do**
  Set $X* = X_{i+1}$
end

4.2.2. Step II: Courier matching

Once the orders have been clustered into batches, the available couriers are assigned to them. Available couriers are prepared to begin their delivery trip immediately upon being informed of their assignment. While solving the matching and routing problem altogether would produce superior results, it would become computationally infeasible as the number of available couriers increases. Therefore, the main goal of this step is to pre-assign couriers to batches of orders based on their likeliness to create efficient routes. The problem is solved by means of an exact formulation based on the following considerations.

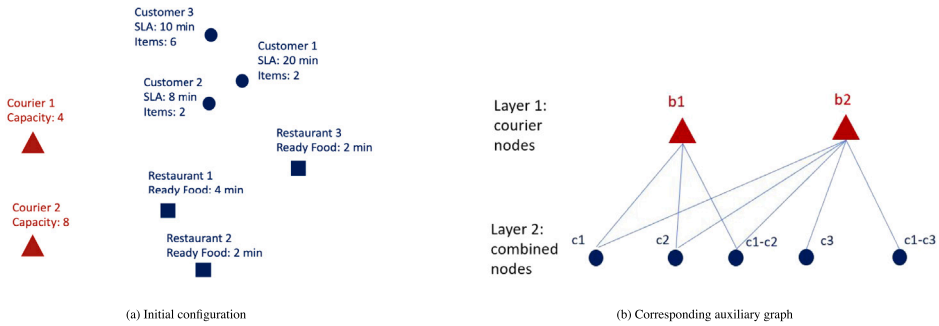


Fig. 4. Example of auxiliary graph for a small cluster of orders and matched couriers.

The assignment of couriers to order batches is determined by batch urgency, demand, and distance. While not explicitly addressed at this stage, the goal is to provide a matching that reduces the first-mile cost and the chance of missing orders when calculating final routes. The provided mixed-integer programming (MIP) formulation (3) addresses these factors. Distance is explicitly considered as the first component of the objective function. Demand and urgency indirectly affect the second component of the objective function. Batches with insufficient couriers will incur a penalty proportional to the potentially unmet demand. Batches' urgency is derived as an inverse normalized function of aggregated urgencies' (available times for assignment and delivery) of their orders. The urgency of each order is determined by the time remaining between the current assignment time and the meal ready time (see Section 5 for further details on the calculation of this value). As a result, batches containing a greater number of 'urgent' orders obtain a higher priority in the pre-assignment (assuming any other feature identical). The ultimate goal of this mechanism is to prioritize the assignment of urgent orders in the subsequent step (Step III) of the algorithm.

The distances between couriers and clusters can be approximated using the courier's average distance to a subset of the cluster's closest restaurants. As it is improbable that a courier will serve the 'farthest' orders within a cluster, especially for large clusters, this should be considered when calculating distances. Therefore, the subset is determined by selecting the closest number of orders. If the cluster contains numerous orders with nearby pickup locations, subset selection becomes redundant. Hence, the size of such a subset depends on the size of its cluster and the average distance between its orders (density). It corresponds to a ratio (adjusted by a generic parameter) of these two variables. Consequently, the distance cost between a courier and a cluster with several distant orders will not be overestimated. The average distance between the courier and such a subset of restaurants is referred to as the distance between the courier and the corresponding subset's centroid (see Fig. 3(b)). The resulting MIP formulation of the problem is described by the set of Eqs. (3).

$$\min_{x} \sum_{i \in D, \chi \in X} (\dot{k}_{i,\chi} x_{i,\chi} + u_{i,\chi}) \tag{3a}$$

$$\text{s.t.} \quad \sum_{\chi \in X} x_{i,\chi} \leq 1, \qquad \forall i \in D, \tag{3b}$$

$$\alpha_{\chi}(w_{\chi} - \rho \sum_{i \in D} x_{i,\chi}) = u_{i,\chi} \qquad \forall x \in X, \tag{3c}$$

$$x_{i,\chi} \in \{0, 1\}, \qquad \forall (i, \chi) \in D \times X, \tag{3d}$$

The objective function aims at minimizing the cost of distance traveled and missed orders. The binary variable $x_{i\chi}$ indicates if the courier i is assigned to a cluster χ. The distance between any courier $d_i \in D$ and the subset of closest orders to d_i in cluster χ is indicated by $\dot{k}_{i\chi}$. The total demand of orders in batch χ is expressed by w_{χ}. The cost of missing orders in a cluster is expressed by the aggregated variable $u_{i\chi}$ and defined according to (3c). Parameter ρ is adopted to approximate the planned level of batching and can be varied according to a priori assumptions. The cluster-specific parameter α_{χ} depends on the relative urgency of its orders and is derived as an inverse normalized function of aggregated urgencies' (available times for assignment and delivery) of their orders. Therefore, clusters characterized by higher numbers of urgent orders will receive higher priority. Constraint (3b) ensures that the couriers are assigned at most once.

4.2.3. Step III: Creating routes by solving several mPDP-TWs

Once the couriers have been assigned to the orders' batches, a mPDP-TW is solved for each batch and its associated couriers. To this end, we adopt a graph-based solution method similarly to Alonso-Mora et al. (2017)'s approach to solve large-scale ride-sharing problems. Here, vehicle routes with pick-ups and drop-offs are abstracted into an auxiliary graph, which allows us to translate the mPDP-TW into an assignment problem, which can be optimally solved by means of an integer linear program (ILP).

In this paper, the auxiliary graph is represented by a bipartite graph $G(N, E)$ where the nodes belong to two independent sets: a set B of matched courier nodes; and a set C of combined nodes. A combined node is created whenever a feasible pick-up-and-drop-off route between a given courier and a subset of order nodes is identified. It represents a combination of pickup and drop-off points that

can be potentially served within the same route. A corresponding link is derived between the courier node and such a combined node with weight equal to the cost of the route. For example, for the small cluster of three orders and two matching couriers illustrated in Fig. 4(a), there would be two corresponding courier nodes and three single order nodes (Fig. 4(b)). We can assume that due to capacity constraints, courier d_1 is only able serve orders o_1 and o_2 whether independently through direct trips, or through a joint route (batch) (o_1, o_2). Therefore, three combined nodes c_1, c_2, c_{12} can be created with three corresponding links to the courier node b_1. Similarly, the other combined nodes and connecting links can be derived by identifying all the feasible routes for courier d_2. In this example, we assume that, due to time constraints it will not be possible to create a combined node c_{123}. In our formulation, the link weights connecting combined nodes to orders nodes correspond to the minimum cost of the pick-up-drop-off sequence involving these orders. Hence, it is possible to exclude other combined nodes (dominated solutions) from the graph without any loss of optimality. Based on this definition of the auxiliary graph, it is straightforward to formulate the mPDP-TW as an assignment problem that can be solved by means of the set of Eqs. (4).

$$\min_{e,y} \sum_{b\in B}\sum_{c\in C'} z_{b,c}\kappa_{b,c} + \sum_{o\in O} y_o u_o \tag{4a}$$

$$\text{s.t.} \quad \sum_{c\in C'} z_{b,c} \le 1, \qquad \forall b \in B, \tag{4b}$$

$$\sum_{b\in B}\sum_{c\in C''} z_{b,c} + y_o = 1, \qquad \forall o \in O, \tag{4c}$$

$$z_{b,c}, y_o \in \{0,1\}, \qquad \forall b \in B, c \in C, o \in O. \tag{4d}$$

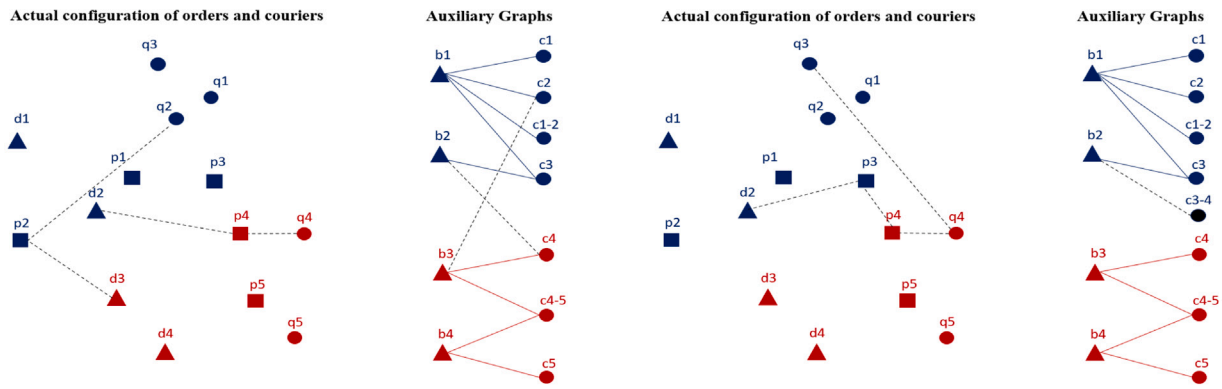
where $z_{b,c}$ corresponds to a binary variable representing the assignment of courier node b to the combined node c. In this formulation, for the sake of simplicity we omit the indices indicating the orders composing the combined node (therefore a node c_{ij} given by batching order i and j is straightforwardly indicated as c). C' represents the subset of combined nodes connected to that courier node, and $\kappa_{b,c}$ denotes the corresponding pick-up-delivery route cost. The binary variable y_o is activated when the order o is not served at a penalty cost u_o. Such cost can be derived as the sum of a constant component and a linear component depending on the urgency of the order. Constraint (4b) enforces the assignment of each courier b to at most one combined node. Constraint (4c) imposes that each order o is either served by one courier connected to a combined node c belonging to the subset C'' of combined nodes containing the order o or missed.

In principle, this method could be used to optimally solve any mPDP-TW. However, even for relatively small clusters, exploring all the possible combinations of couriers and orders would become computationally expensive for batch sizes greater than two orders. Alonso-Mora et al. (2017) obviate this issue by using heuristics to identify promising combinations of pick-up-drop-off routes and by introducing time-based termination criteria to this exploratory procedure.

In our study, we adopt a restricted dynamic programming (RDP) approach to efficiently derive PDP routes and translate them into combined nodes and corresponding links. Here, potential sequences of stops are incrementally derived based on partial solutions (states, σ). This approach, which entails accessing previously computed partial solutions (shorter sequences of pick-up and drop-off points) and discarding unfeasible and sub-optimal sequences during the process, allows for significant computational savings compared to a vanilla enumeration of feasible solutions. By introducing a threshold of maximum explored states per iteration (of the RDP algorithm), the least performing partial solutions are discarded from the computation. This method allows us to optimally solve the assignment and batching problem for clusters of limited size, while at the same time, identifying high-performing solutions for larger clusters. Similarly to Desrosiers et al. (1988), a series of feasibility checks are performed at each iteration (of the RDP algorithm) in order to discard solutions that violate any capacity, order, maximum size, and time constraint. Furthermore, dominated solutions (routes characterized by sub-optimal sequences of stops) can be identified and discarded throughout the computation in order to limit the number of partial solutions analyzed per iteration.

Finally, in order to deal with clusters characterized by pre-assigned couriers, we introduce an approximation based on the couriers' features to further reduce the algorithm's computational burden. Couriers that are very similar to each other in terms of location, capacity, and speed are considered as single higher-degree 'super-couriers' within the route construction process. Rather than generating routes for each of them directly, a 'super-courier' enables the generation of 'aggregated' nodes and links in the auxiliary network that can be duplicated later when solving the corresponding assignment problem (final phase of Algorithm 3). Once the corresponding links are derived for super-couriers in the auxiliary graph, these can be quickly duplicated for the corresponding similar couriers. A duplicate link corresponding to an actual courier's route will be created by modifying the actual first mile costs of the super-courier's route. Sets of similar couriers can be determined by identifying groups of couriers within a time range (t_d) from each other. For example, if the two couriers matched to the same cluster of orders in Fig. 4 are located within a relative small distance from each other and they have the same speed and capacity characteristics, it is sufficient to explicitly derive routes only for one super-courier. A representative super-courier is chosen such that its distance from the cluster's orders is the largest among similar-courier in order to avoid the creation of unfeasible solutions during the duplication procedure. The choice of the proximity threshold depends on the trade-off between the resulting computational cost and the accuracy of the obtained solution (see Section 5 for further details on the effects of different threshold ranges). As highlighted in Section 5, this solution leads to major computational savings.

The methodological approach described above is summarized in Algorithm 3. In the initial step of the algorithm, a subset of super-couriers is selected based on couriers' respective distances (lines 2–7). Only couriers within a given threshold time distance become candidates for the aggregation into super-couriers (with a higher time threshold resulting in a higher level of approximation).



(a) Creation of edges by identifying new assignments

(b) Creation of nodes and corresponding edges by identifying new batches

Fig. 5. Local search moves.

Then, alternative routes can be progressively created by means of RDP for each super-courier (lines 13–38). The RDP procedure relies on the concept of state σ corresponding to a sequence of visited nodes. Each state can be defined by an ending node j and a predecessor state (which store all the information about the previously visited nodes). The RDP procedure is initialized by choosing the super-courier as a first node and proceeds by iteratively expanding existing states, where an unvisited node is added as terminal node. For each state σ, the corresponding cumulative cost $k(\sigma)$ and cumulative time $t(\sigma)$ can be recursively derived from its predecessor plus the additional leg to the terminal node. Before a new state σ' is created by expanding σ with a node j, a series of feasibility checks (line 23) is performed to ensure compliance with capacity (a), maximum batch size (b), time-windows (c), and pick-up-drop-off precedence constraints (d). In addition to that, a quick scan procedure is performed to verify that all planned drop-off nodes (according to already visited pick-up nodes) can be visited without violating time constraints (e). These checks are crucial to limit the size of the problem from growing exponentially.

In order to reduce the computational effort, dominated states can be identified and removed (lines 25–38) without compromising the optimality of the solution (Bellman's Principle of Optimality). A dominated state can be identified when another state exists with the same terminal node j and visited nodes, which however exhibits a lower cumulative cost and time. Along with the RDP procedure, combined nodes and corresponding links are added to the auxiliary graph each time the state's terminal node corresponds to a drop-off node and all the orders in the sequence are served (lines 25–35). Since in the RDP procedure only the most promising states are considered for the following iteration, at this stage of the algorithm (lines 36–37), it is possible to set a maximum number of states ν for the next iteration of nodes scanning (RDP). Based on the size of the cluster and threshold value chosen, this artifact can yield significant efficiency gains at relatively low expense (see Section 5.3). In the final part (lines 39–42), the algorithm leverages the concept of super-couriers to duplicate previously created link (of the auxiliary graph) for those couriers whose routes were not explicitly computed through the RDP. The costs of the links can be adjusted based on their relative positions in relation to the corresponding super-courier.

4.2.4. Step IV: Local search and recombine partial solutions

At this stage of the proposed OBA algorithm, the clusters' solutions obtained at the end of Step III are combined and improved by identifying new, potential alternative batches of orders and assignments to couriers. The main idea is to 'restore connections' by creating combined nodes and links between the previously created auxiliary graphs and re-solve the mPDP-TW as an assignment problem for a global auxiliary graph. In order to do that, LS (Algorithm 4) is performed in two main directions: (i) creating alternative assignments for previously created potential batches (Fig. 5(a)), and (ii) creating alternative batches from orders belonging to different clusters (Fig. 5(b)). Both operations involve creating combinations that were not explored in the third step of the OBA algorithm.

First, new potential assignments between couriers and orders or batches are created by scanning previously created nodes for each auxiliary graph (lines 3–6). For each combined node, the first-mile cost (cost of reaching the first restaurant in the route), is computed for those couriers belonging to different clusters. The weight of the corresponding new link can be straightforwardly derived by replacing the original first-mile cost with the newly obtained one. Computationally, this is a relatively inexpensive procedure. In order to limit the number of new edges (to keep the size of the final graph manageable), only the subset of the cheapest ψ new connections is considered (lines 7–9).

Then, new potential batches of orders belonging to different clusters are created by analyzing potential inter-cluster combinations (lines 10–18). In order to identify such new favorable solutions, a new similarity measure between pairs of orders is introduced to

Algorithm 3: RDP Algorithm with Super-Couriers for the auxiliary graph development

```
Inputs: Set of orders O, Set of couriers D; Time threshold for Super-couriers t_d, Max. number of states v;
for all couriers d_i ∈ D do
    Identify subset J_i of couriers d_j within threshold time distance from courier d_i: (t_ij < t_d) ;
    Create empty Set F of discarded couriers for the RDP ;
end
Create empty Set D* of super-couriers;
Sort subsets J_i by decreasing size;
for all subsets J_i do
    if d_i ∉ F then
        Add courier d_i to set D* and add similar couriers d_j to set F;
    end
end
for all super-couriers d*_i ∈ D * do
    for all orders o_i ∈ O do
        Derive corresponding pick-up node p_i and add it to the pick-up node set P;
        Derive corresponding drop-off node q_i and add it to the drop-off node set Q ;
    end
    Derive Set of Nodes N = P ∪ Q ;
    Set super-courier location as origin node, add corresponding state σ_0 to the set of newly created states Σ ;
    while |Σ| > 0 do
        For each state σ in Σ, scan each node j not contained in the state's visited nodes;
        Derive cost k(σ') and time t(σ') of the new state σ' based on its predecessor as σ' = σ + j ;
        Perform feasibility checks for potential new state σ': (a) capacity, (b) maximum batch size, (c) time-windows, (d)
          pick-up-drop-off precedence, (e) reachability of drop-off nodes;
        if feasibility checks are positive then
            Add new state σ' to the temporary set of newly created states Σ'
        end
    end
    for each set Σ_j ⊂ Σ' of states with the same terminal node j do
        Sort states σ ∈ Σ_j by cost;
        Filter dominated solutions and update Σ_j;
        if j ∈ Q then
            Identify state with minimum cost σ_m in Σ_j;
            if All pick-up nodes p_i in σ_m have their corresponding drop-off node q_i in σ_m then
                Add Combined Node c_i to the auxiliary graph G ;
                Add corresponding link e_{b*_{c_i}} to the auxiliary graph G;
            end
        end
    end
    Update Σ = ⋃ Σ_j ;
    Update Σ by selecting the v cheapest states ;
end
for courier f ∈ F do
    Find corresponding super-courier d* ;
    Create duplicate links e_fc based on e_{b*_c} ;
end
```

leverage all the information obtained in the previous steps of the main algorithm. The new similarity between two random orders, Order 1 and Order 2 (s'_{12}), is given by

$$s'_{12} = \frac{s_{12}}{k_1 \cdot k_2 \cdot l_{12}}, \tag{5}$$

where s_{12} corresponds to the similarity measure between Order 1 and Order 2 derived in Step II of the OBA algorithm (Algorithm 1). Further, k_1 and k_2 represent the respective orders' cost according to the solution identified at the end of Step III of the OBA algorithm, and l_{12} is a binary variable equal to 1 if the two orders belong to different clusters and an arbitrarily high number otherwise. Based on this new measure, for each order, it is possible to select a set of ψ' most promising orders for the creation of new batches. For those candidate orders belonging to the same cluster, a best insertion procedure is performed for the order

and subsets of candidates (lines 12–18), by considering their previously created routes as pivots. A pivot route is a route where the original sequence of drop-off and pick up stops is unaltered. This procedure can be performed incrementally by inserting the order in routes of increasing size, until feasibility constraints are violated. Finally, a 'merging move' is performed to derive routes given by the combinations of orders from three or more different clusters (lines 22–27). The combination of three or more orders explored are given by the ψ'' combinations with the lowest average new similarity score. For these, a simple insertion procedure is performed to create new batches. Clearly, the higher values are chosen for the LS moves, the more potential solutions are explored with a growth a computational times (see Section 5.3 for further details). Once new combined nodes and links obtained from the LS moves are added to the global auxiliary graph, the final batches and assignment are derived by solving the mPDP-TW as an assignment problem like in Step III (line 28).

Algorithm 4: Local Search and Re-combine Algorithm

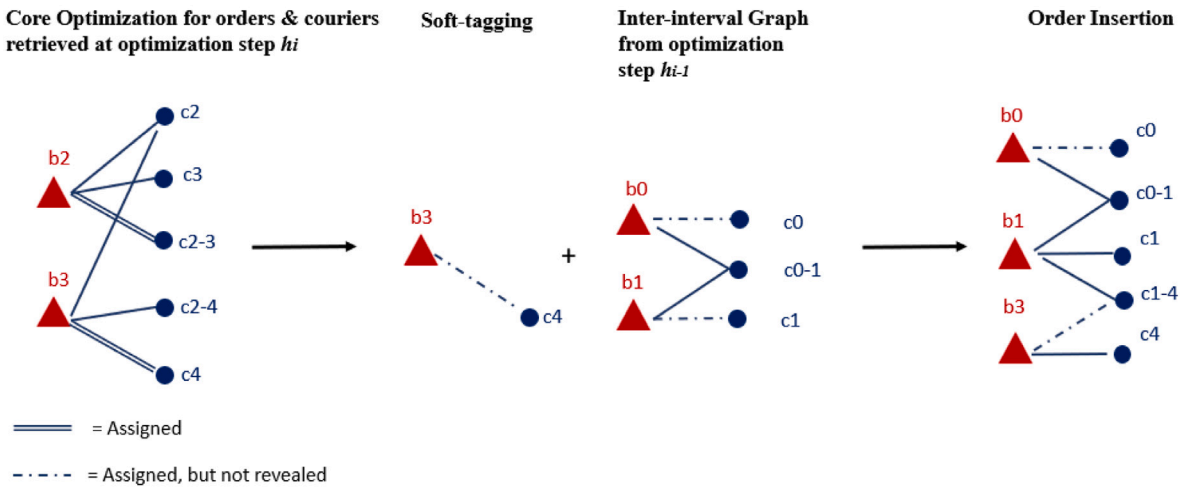
1 **Inputs:** auxiliary graphs G_i, Set of orders O, Set of couriers D, threshold time t_d, threshold parameters for LS moves ψ, ψ', ψ'' ;
2 Merge auxiliary graphs into a global auxiliary graph $G = \bigcup G_i$;
3 **for** *each combined node* $c \in C$ **do**
4 Identify courier nodes b such that $\nexists e_{bc} \in G$ and $k^{\cdot}_{bj} < t_d$ where j is the first restaurant visited in c ;
5 Create corresponding edges e_{bc};
6 **end**
7 **for** *each courier node* $b \in B$ **do**
8 Add ψ cheapest newly created edges e_{bc} to G;
9 **end**
10 Derive new similarity matrix S' for all orders' pairs according to Equation (5);
11 Create set M of size ψ' of potential matches m with the lowest new similarity score s';
12 **for** *each order* $o \in O$ **do**
13 Create subset M_o of potential matches m containing order o and label them by original cluster χ of belonging;
14 Create pivots of orders belonging to the same cluster with existing corresponding combined node c^*;
15 Perform best insertion of order o into pivots c^*;
16 **if** *the new batch is feasible* **then**
17 Create new combined node c_{om} and edge $e_{dc_{om}}$;
18 Add c_{om} and $e_{dc_{om}}$ to G;
19 **end**
20 **end**
21 Create set M' of size ψ'' of potential batches m' larger than 3, with the lowest average new similarity score;
22 **for** *each potential batch* $m' \in M'$ **do**
23 Derive corresponding pickup-dropoff route by insertion algorithm; **if** *the new batch is feasible* **then**
24 Create corresponding combined node $c_{om'}$ and edge $e_{dc_{om'}}$;
25 Add $c_{om'}$ and $e_{dc_{om'}}$ to G;
26 **end**
27 **end**
28 Solve mPDP-TW as an assignment problem for G according to Equations (4)

4.2.5. Step V: Integration of advanced policies

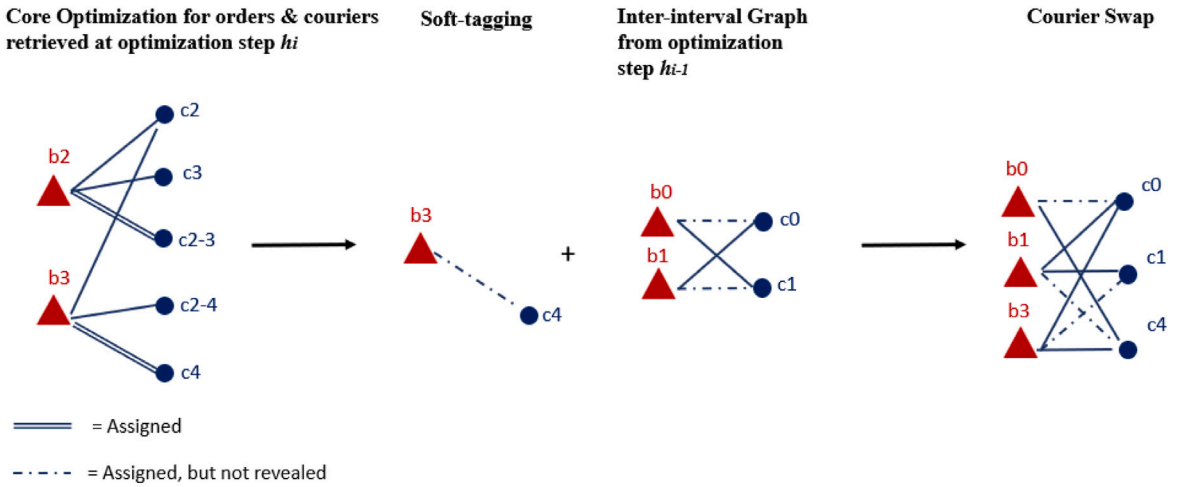
The approach described in the previous sections can be straightforwardly employed to solve a Greedy Policy approach. On the other hand, it can also be extended to incorporate two OBA 'advanced policies', involving a partial re-optimization of orders/couriers throughout multiple time intervals: the *Swap* policy, and the *Insertion* policy. Both policies rely on the concept of 'soft-tagging'. A batch identified at the end of Step IV can be soft-tagged by postponing the communication to the courier if the pickup at the restaurant (which partly depends on the food preparation time) is not delayed. Therefore, soft-tagged orders (or batches) are revealed to their assigned couriers after a specific time during the simulation. The purpose of this practice is twofold: reducing the courier's wait at the restaurant; and allowing partial or complete modification of soft-tagged batches in following optimization steps. In real-world applications, soft-tagging has some potential drawbacks: the soft-tagged courier could log out, travel times might be longer than the estimated one (ultimately resulting in a delay). In order to reduce such risks, a threshold time, or just-in-time (JIT) parameter, is set to decide whether to reveal or not a batch. If the estimated wait time at the restaurant is above the JIT parameter, the order will not be revealed until the wait time reaches this threshold. The chosen value of the JIT parameter clearly affects the extent of re-optimization occurring throughout multiple intervals.

The soft-tagging procedure is the basis for implementing the two proposed advanced OBA policies. In the *Insertion* policy, soft-tagged orders obtained from the current optimization step h_i can be inserted into soft-tagged batches obtained from the previous optimization epochs (h_{i-1}, h_{i-2},...). Similarly, in the *Swap* policy, soft-tagged batches can be swapped between the assigned couriers. Idle couriers who became available over the last optimization interval are also considered to improve the solutions.

Both advanced policies leverage the graph representation described in Section 4.2.3 to improve the optimization efficiency. The concept is briefly illustrated in Fig. 6(a)–Fig. 6(b). Once the soft-tagged orders (and corresponding couriers) are obtained at the end of



(a) Insertion policy process



(b) Swap policy process

Fig. 6. Conceptualization of advanced policies.

the core optimization algorithm, their corresponding auxiliary graph is merged with the graph corresponding to soft-tagged batches derived in previous intervals. Such a graph is referred to as an 'Inter-interval Graph' (I) since it is retained throughout different optimization steps and updated based on the JIT parameter (γ). In the example, at the optimization step h_i, the OBA algorithm has generated an auxiliary graph for two idle couriers d_1, d_2, and three newly placed orders, o_2, o_3, o_4. As a result of soft-tagging, the assignment (d_3, c_4) is not revealed. At this stage, based on the chosen advanced policy, the corresponding graph is merged differently with the Inter-interval graph obtained from the previous optimization steps. From a 'graph perspective', the *Insertion* policy focuses on creating new combined nodes (and corresponding links), whereas the *Swap* policy focuses on developing new links. Algorithm 5 summarizes this process for a generic optimization step h_i where newly arrived orders O and idle couriers D have been considered. The algorithm also develops and updates the Inter-interval Graph structure. The first step of the algorithm consists of soft-tagging orders (and corresponding assigned couriers) and updating the Inter-interval Graph inherited from the previous optimization step. Such update is necessary to account for assignments that might have been 'revealed' meanwhile. Once the new Inter-interval Graph structure is updated, in the Courier Swap policy, new links are added to the original graph by scanning feasible replacements among soft-tagged pairings of couriers-orders. In order to limit the computational costs arising with a growth of the graph, new links with costs can be added based on an arbitrary threshold. Similarly, in the Order Insertion policy, new links (and corresponding nodes) are created by scanning feasible order insertions into existing soft-tagged routes.

Algorithm 5: Advanced policies algorithm at optimization step h_i

```
Inputs: Set of orders O, Set of couriers D; JIT parameter γ; auxiliary graph G; Set of temporary solutions obtained at
  the end of Step IV Z; Inter-interval graph I;
for each temporary solution z_dc ∈ Y do
    Soft-tag depending on the travel time to the first order i of combined node c and meal preparation time t_r:
    if (t_r − t_ij) > γ then
        Add d_i to the set of soft-tagged couriers D′;
        Add o_j to the set of soft-tagged orders O′;
    end
end
for each edge e_bc ∈ I do
    if If corresponding courier d and orders o are revealed then
        Remove e_bc from I;
    end
end
for each edge e_bc ∈ G do
    if If corresponding courier d and orders o are soft-tagged then
        Add e_bc to I;
    end
end
if Swap Policy is activated then
    for each edge e_bc ∈ I do
        for each soft-tagged courier d′ in D′ do
            if If corresponding edge e_b′c ∉ I then
                Derive first mile cost k_d′i;
                Perform feasibility checks and add edge e_b′c to graph I
            end
        end
    end
end
if Insertion Policy is activated then
    for each combination of soft-tagged orders (o_i, o_j) ∈ O′ do
        if corresponding combined node c_ij ∉ I then
            for each edge e_bc connected to combined node c_i in I do
                Insert order j and derive new edge e_dc_ij cost;
                Perform feasibility checks, and add edge and corresponding combined node c to graph I;
            end
        end
    end
end
Solve mPDP-TW as an assignment problem for I according to Equations (4)
```

5. Analysis

To demonstrate the effectiveness of our proposed heuristic technique, the performance of the solution approach is evaluated for a wide range of demand and supply conditions (Section 5.2). The analyses continue with an in-depth evaluation of the influence of various approximations used throughout the steps of the algorithm on the algorithm's performance (Section 5.3). We conclude with an evaluation of the impact of various policies on the quality of the obtained solutions (Section 5.4).

5.1. Problem instances

In order to generate realistic problem instances, we create realistically sized problem instances informed by actual historical order and courier data from one of India's leading ODFD platforms, referred to as *The Company* throughout the study. The Company's business operations analyzed consist of order and courier data collected between May and June 2019 for the city of Bangalore (India). Four characteristic demand time periods (9 am–11 am, 12 pm–2 pm, 4 pm–6pm, and 7 pm–9 pm) are analyzed for the south-eastern region of the city including major neighborhoods, such as Koramangala, Byrasandra–Tavarekere–Madivala (BTM), and Hosur-Sarjapur Road (HSR) (see Fig. 7).

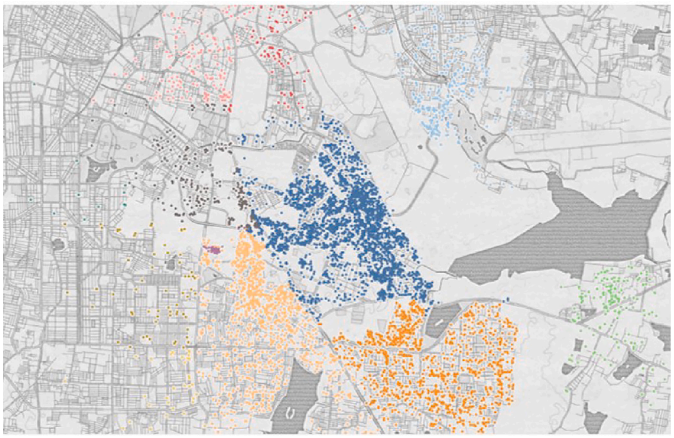


Fig. 7. Overview of the reference real-world scenario adopted for creating instances.

Due to the large demand volumes and geographical extent, The Company's platform operations are run independently for different sub-regions of Bangalore. In this study, we consider an area of around 20 km^2 with an average of 8,000 to 10,000 orders depending on the day of the week. In this study, a set of 20 instances (Instance 1–20) characterized by two levels of demand and supply intensity ('balanced demand-supply and 'undersupply') is created. In the first group, demand is generated with an intensity of 1000 orders/h, and supply corresponds to an average fleet size of 400 couriers (considering logins and logouts). In the second group, a fleet of 200 couriers is operated to serve the same level of demand. From a demand perspective, orders are randomly created such that their last-mile distance, corresponding SLA, and size (number of items) are drawn from uniform distributions representative of the analyzed real data. From a supply perspective, identical couriers (in terms of capacity, speed, and loading/unloading efficiency) are randomly generated on the network. All couriers are considered idle when logging into the system. Additional information on how the problem instances were generated can be found in Appendix B. The input parameters for the two-hour simulation are summarized in Table B.4.

In order to investigate more in detail the performance of the heuristic, the effect of different assumptions, and choice of parameters, 'static instances' representing a single re-optimization epoch are also analyzed (Instance 21–68). Based on a hierarchical clustering procedure, three possible levels of overall demand intensity (20, 50, 100 orders placed per 2-min interval), shares of urgent orders (0%, 50%, 80% of orders with meal preparation times below 10 min), and supply availability (courier-to-order ratios of 0.5, 0.7 and 1.0) are considered. In order to create a more standardized scenario a Manhattan grid-network characterized by regular blocks of 100 meters is created. Table B.3 provides a detailed description of this set of instances.

As mentioned in Section 2, a distinctive feature of the OBA problem for ODFD platforms consists of the uncertainty in the supply side. Since the platform operator relies on crowd-sourced couriers, the platform capacity can fluctuate between optimization steps. In order to assess the ability of our proposed OBA algorithm to manage unfavorable conditions characterized by over-demand, we investigate its performance under 'high stress conditions'. A situation of high stress can occur when the number of orders placed during the last optimization step is high compared to the number of available couriers while order preparation times are low. To reflect this, an additional set of instances (Instance 69–100) characterized by different levels of courier-to-order ratios and urgency of orders is created (see Table B.5). For this analysis, urgent orders are considered to have preparation times of less than 10 min (cf., Step II and III). The penalty cost for postponing an order assignment is characterized by a fixed component equal to 100 units and a variable component given by the product of 10 units and the difference between the meal preparation time and the urgency threshold. Both the urgency threshold and the penalty cost reflect the platform operator's willingness to risk postponing orders (additional investigations on this managerial aspect are included in Section 5.3). In total, 100 instances are generated (all of which are available as an electronic supplement to this paper).

5.2. Performance analyses

5.2.1. Heuristic performance compared to exact method

In order to investigate the quality of the proposed heuristic, we compare its results with those obtained by solving the problem as a mPDP-TW based on the original MIP formulation. Preliminary tests have shown that, with a standard work computer (in our case with an i7-8665U CPU at 2.11 GHz), solving instances larger than 20 customers to (near) optimality requires at best several hours. Hence, for practical reasons, we focus here on solving only the small instances (Instance 21–36) to optimality.

In Table 1, the performance of our proposed heuristic is reported for instances characterized by different courier-to-order ratios and order densities, in comparison to the optimal solutions. The chosen delivery costs for the optimization problem correspond to a courier waiting fee of 1 unit/min, a travel cost of 6 unit/km, and a flat fee of 5 units. The maximum batch size is set to 4 orders. The chosen parameters for the heuristic (maximum cluster size of 10 orders, threshold time for aggregation in super-couriers equal to

Table 1
Heuristic performance for static instances (Instance 21–36)

Instance	Optimal			Heuristic			gap (%)
	Avg. cost (per order)	Batching (%)	Avg. batch size	Avg. cost (per order)	Batching (%)	Avg. batch size	
21	17.58	90	2.00	17.76	90	2.00	1.00
22	19.38	70	2.00	19.99	70	2.00	3.17
23	17.89	90	2.00	18.05	90	2.00	0.90
24	19.13	80	2.00	19.74	80	2.00	3.21
25	18.64	80	2.00	19.36	70	2.00	3.86
26	18.18	80	2.00	19.05	70	2.00	4.80
27	17.92	90	2.00	18.62	90	2.00	3.87
28	18.23	90	2.00	18.36	80	2.00	0.74
29	16.84	90	2.00	17.39	90	2.00	3.24
30	15.78	95	2.11	15.91	95	2.11	0.85
31	16.25	95	2.11	16.29	95	2.11	0.25
32	17.39	85	2.13	17.92	90	2.00	3.06
33	16.09	90	2.25	16.52	90	2.25	2.66
34	17.32	90	2.00	18.53	80	2.00	6.98
35	16.55	95	2.11	16.61	95	2.11	0.37
36	16.66	80	2.29	17.58	85	2.13	5.50

2 min, and LS parameters) represents a good compromise between performance of the algorithm and computational effort required. Further discussion on the influence of different heuristic settings is provided in Section 5.3.

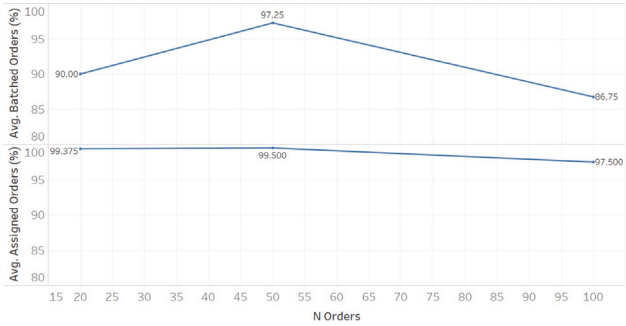
The computational effort required to solve problem instances 21 through 36 to optimality varies between 1000 and 5000 s depending on the demand and supply configuration. The computation time of our proposed heuristic on the same instances, varies between 39 and 73 s depending on the configuration of couriers and customers, with an average time of 47 s. The results shown in Table 1 indicate that for these relatively small instances, the proposed heuristic obtains results that lie within 2.8% on average from the optimal solution of the OBA problem. In all the analyzed instances, the heuristic approach was able to obtain batching rates within 10% from the optimal levels. It is interesting to see that the quality of the heuristic seems independent from the courier-to-order ratio, as the average gap does not increase for different levels of supply.

For larger problem instances (e.g., 50 and 100 orders), the results of our proposed heuristic (with optimized parameters and a run time threshold of 600 s) are analyzed by considering several performance metrics. We summarize the results of our experiments on larger problem instances (Instance 37–68) in Fig. 8. The results obtained for these instances are in line with those obtained for the smaller ones. As expected, the average cost per delivery tends to increase in the instance size due to the sub-optimality of the heuristic solution. However, the increase is limited, particularly for medium-sized instances of 50 orders, where the average increase in cost per delivery is around 6%. The performance deterioration of the algorithm can be mainly attributed to a decline in the overall level of batching (from 90% to 86%) and a reduction of the average batch size (from 2.1 to 2.0 orders). This seems an acceptable compromise between accuracy and computational effort. In terms of order assignment, the number of missed/postponed orders slightly increases to 2.5% for large problem instances. It is unclear whether this is due to an increase in the probability of having 'unserviceable' orders as the problem instance size grows, or due to a limitation of the heuristic itself.

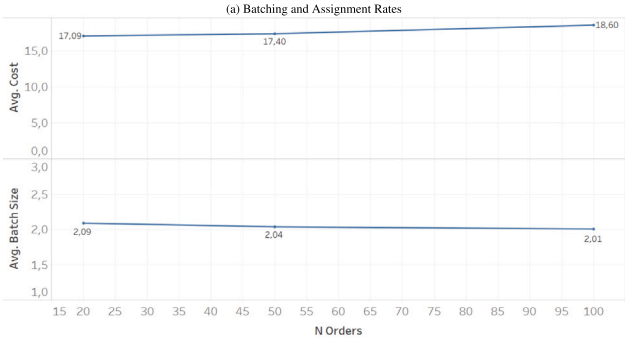
The results of the more realistic and dynamic scenarios described by Instance 1–10 are similar. The performance results corresponding to different policies summarized in Fig. 9. As expected, the average costs and batch size obtained for the myopic policy are similar to those obtained in the larger static instances investigated (50–100 orders). Clearly, both advanced policies can achieve considerable improvements of the OBA solutions. The Order Insertion and Courier Swap respectively reduce the average delivery cost by about 15% and 10%. As Fig. 10 illustrates, the improved OBA efficiency of the two advanced policies correspond to a growth of the batch size and increased number of batched orders. As expected, the benefits of both policies come at a certain cost in terms of increased computations. Computation times are on average 100% larger than in the Insertion policy and 30% larger in the Swap policy.

In the case of high stress instances, for relatively small sizes, the algorithm performance is also assessed based on the corresponding optimal solution. As seen in Table C.6, the suggested heuristic produces solutions that are extremely near to the optimal. Indicators like the percentage of postponed orders and the average slack time for the postponed orders (i.e., the difference between meal preparation time and the length of the optimization time step) indicate a robust performance of our proposed heuristic even in high stress conditions. This result indicates that the heuristic is capable of identifying the well-suited orders for assignment postponing. The batches resulting from our heuristic are also very similar to the ones obtained observed in the optimal solutions with similar prioritization of orders and batch costs. The corresponding average optimality gap of the heuristic solutions is less than 3%. Only for two instances a gap larger than 5% was observed. Interestingly, the algorithm performance does not seem to be significantly affected by the amount of urgent orders, nor by the demand-supply ratio.

The results obtained for large-sized high-stress instances (instances 85 through 100) are in line with those obtained for smaller sizes, according to the several metrics summarized in Fig. C.17. The average level of postponed orders only increases by 1%–2%, while the average slack time increases by about 1 min. As expected, larger instances are characterized by a cost increase (on average by 3%) due to the approximations adopted to limit the computational effort. Interestingly, when comparing instances with 20 orders



(a) Batching and Assignment Rates



(b) Batch Size and Cost

Fig. 8. Heuristic performance for different levels of orders (Instance 21–68).

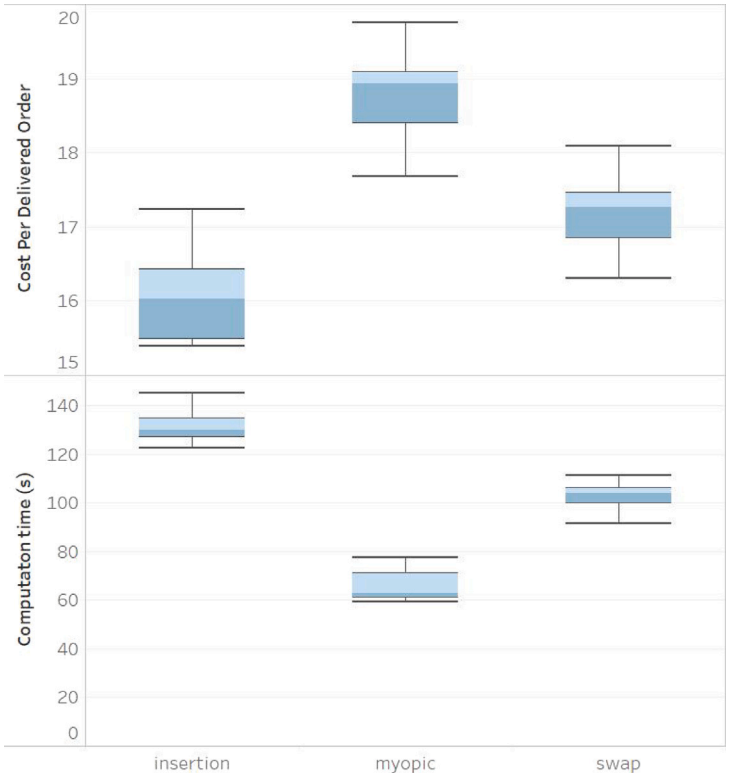


Fig. 9. Heuristic performance for dynamic instances (Instance 1–10).

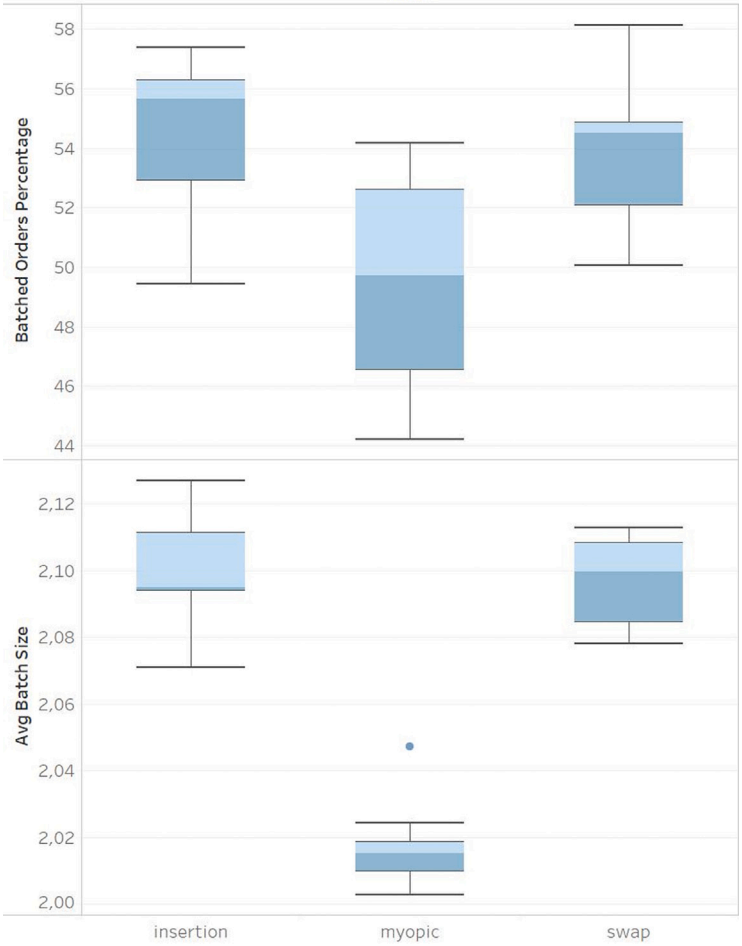


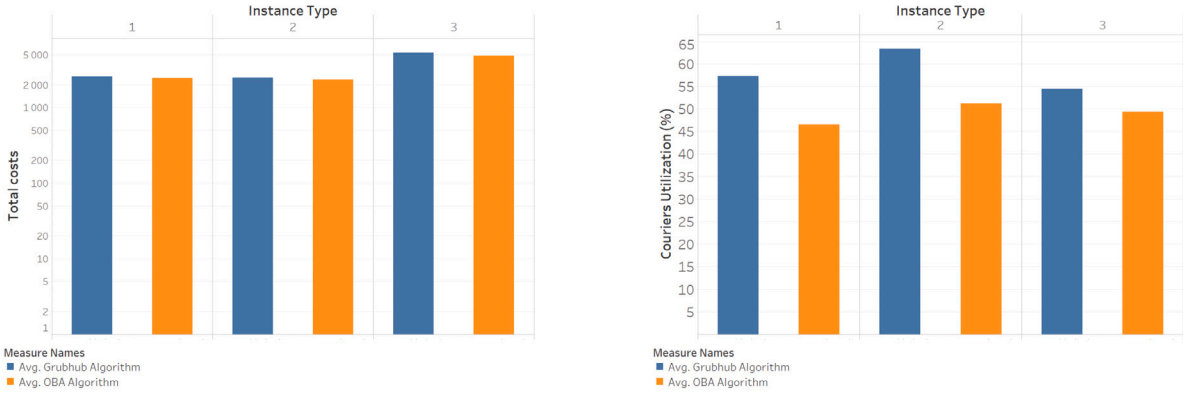
Fig. 10. Batching performance for dynamic instances (Instance 1–10).

against 100 orders, the growth of delivery costs is limited for cases with both low density and courier-to-order ratios (less than 1%) and high density and courier-to-order (C2O) ratios (around 40%). This phenomenon could be explained by the relatively small growth of the solution space for the first category of instances. Overall, these results suggest that the proposed OBA algorithm performs well in high-stress conditions for all problem instance sizes.

The results are in line with those obtained in dynamic problems where delaying order assignment over different optimization epochs is necessary due to demand and supply fluctuations. A more detailed analysis of the postponement mechanism for dynamic instances confirms shows that delayed assignments decrease linearly as the C2O ratio increases (see Fig. C.18). This behavior is confirmed by the average slack times of delayed orders for high C2O ratios. Most delayed orders have slack time of 600 s, corresponding to the threshold of urgency set in our experiments.

5.2.2. Performance comparison to benchmarks in the literature

To further investigate the capabilities of the proposed algorithm, we run an additional round of tests with an existing set of instances created by Reyes et al. (2018) and further investigated by Yildiz and Savelsbergh (2019). These instances reproduce historical daily order and courier patterns of the delivery company Grubhub in a realistic scenario. We select sixteen instances corresponding to three alternative configurations characterized by different sizes of orders (242 to 505), restaurants (54 to 116), and couriers (61 to 113). In our tests, the orders are characterized by the same pickup and drop-off times, whereas their promised times are equal to the 'click-to-door targets.' While both the algorithm of Yildiz and Savelsbergh (2019) and our OBA algorithm address the meal delivery problem (where the service provider minimizes its costs in dynamic settings), they present some crucial differences in their modeling and optimization. In the Grubhub's problem, the fleet is composed of two types of couriers: 'delivery-only' ones, who are paid a fixed amount for each order delivered, and 'minimum-pay' ones, who are guaranteed a minimum pay for their shift. In addition, it does not consider the underlying road network when calculating distances between points. The objective function is formulated accordingly as a linear program where work packages representing trips or bundles are selected to minimize the overall cost of employing couriers. Given these differences, it is not possible to directly benchmark our algorithm with the one



(a) Benchmarking of costs for Grubhub instances

(b) Benchmarking of computation time for Grubhub instances

Fig. 11. Performance comparison with Yildiz and Savelsbergh (2019).

of Yildiz and Savelsbergh (2019). Nevertheless, it is possible to calculate the costs of our results based on Grubhub's function in the post-processing stage to gauge a better understanding of our algorithm's performance. In Fig. 11, we illustrate the performance of the two algorithms. The overall costs calculated based on Grubhub's function are close (Fig. 11(a)). Our OBA algorithm determines on average 6.1% lower total costs. Although our optimization algorithm optimizes according to a different objective function, it manages to achieve a reduction in the overall costs due to higher batching levels (the formulation of Yildiz and Savelsbergh (2019) allows batching only if orders are from the same restaurant). This hypothesis is confirmed by the resulting courier's utilization levels (Fig. 11(b)), which are consistently lower (between 7% and 31%) than those obtained by Yildiz and Savelsbergh (2019). The average computation time varies between 16 and 42 s depending on the problem size. Additional details about our algorithm's performance for the Grubhub instances are reported in Appendix C.

5.3. Algorithmic insights

The OBA algorithm proposed in Section 4 relies on a sequence of techniques for decreasing the computational effort and allowing real-world applications. The first three steps of the algorithm add some approximation and, as a consequence, a potential degradation in the performance of the resulting batches and assignments. In this section, we examine the sensitivity of the OBA algorithm to different components (Step I, Step III, and Step IV) of the framework and investigate the trade-off between solution quality and computational efficiency. The tests are carried out for problem instances 21 through 68 described in Section 5.1, which include configurations with different levels of demand (20 to 100 orders), supply (10 to 100 couriers), and spatial density (4 km^2 to 20 km^2).

5.3.1. Impact of clustering (Step I)

The clustering approach implemented in Step I (see Section 4.2.1) prevents the OBA algorithm from tackling a computationally expensive mPDP-TWs instance all at once and instead divides the problem into sub-problems, hence allowing for the solution of large problem instances. Breaking down the original problem into several smaller mPDP-TWs, on the other hand, reduces the number of explored potential OBA solutions since Step IV can only partially compensate for the batching and assignment opportunities that were 'missed' due to clustering (see Section 4.2.4). Fig. 12 illustrates the trade-off between increasing the maximum allowed cluster size and reducing the computational efficiency of the OBA algorithm. The effect of decreasing the chosen maximum cluster size are analyzed for different sizes of the OBA problem. These input changes are expressed by means of the 'cluster ratio', a ratio between the cluster size and the overall number of orders. Due to computational constraints, instances of size 50 and 100 are limited to a maximum cluster size of 25 and 30 orders, respectively. Tests were conducted by decreasing the maximum cluster size up to 5 orders, for all instances. As expected, increasing clustering leads to a deterioration of the solution quality. However, regardless of the instance size, the overall increase in the solution value (i.e., cost) does not exceed 2%. Interestingly, for large and medium-sized problem instances (100 and 50 orders), the deterioration rate seems to decrease once cluster ratios of 0.2–0.25 are reached. For these values, the computational effort can be greatly reduced by 40%–50% in comparison to higher cluster ratios of 0.4–0.5. This result suggests that there might be a 'sweet spot' for clustering, which allows significant computational benefits for relatively low performance losses. From a performance perspective, small sized instances (20 orders) follow a similar trend, with a minor decrease up to 2%. Interestingly, no computational improvements were measured for decreasing cluster ratios. However, it is worth mentioning that for this instance size the OBA algorithm's computation times are already very short (47 s on average).

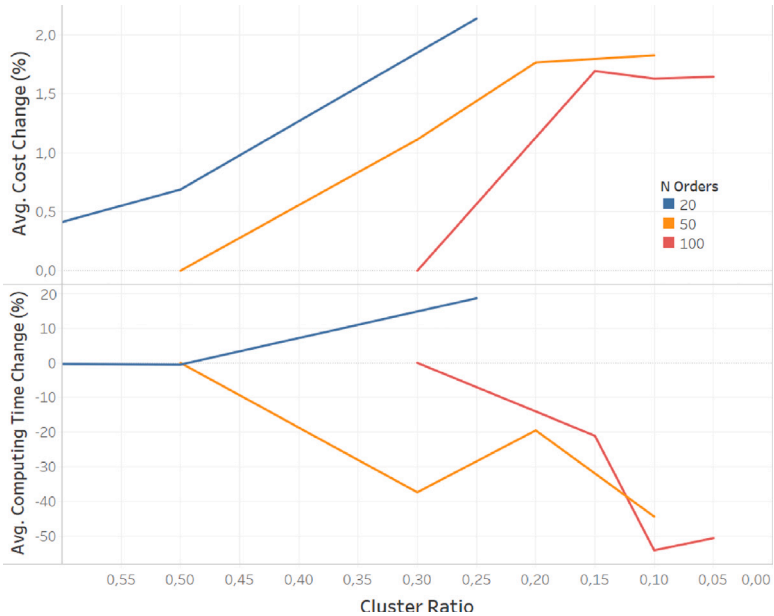


Fig. 12. Clustering impacts overview.

5.3.2. *Impact of super-couriers (Step II)*

Identifying super-couriers that represent several 'actual' couriers avoids considerable duplication efforts that would be spent in the dynamic programming procedure of Step III. Since we select couriers with the longest first-mile distances in the procedure of defining super-couriers for precise route computations (to prevent the development of infeasible solutions), greater approximations determine a reduction of the examined potential solutions (see Section 4.2.3 for further details). Fig. 13 illustrates the results of different threshold time distances τ adopted for the aggregation of couriers into super-couriers (60, 180, and 360 s). While using larger values decreases the total processing time required by the OBA algorithm, it also increases the average cost per delivery, as predicted. However, the cost increase is minor compared to the substantial savings in computing time obtained. Small approximations (60 and 180 s) determine considerable computing gains for big and medium-sized problem instances (around 10% and 25%). The equivalent increases in the average cost per delivery are 0.1% and 1.5%, respectively. Allowing for higher approximations (360 s) does not result in significant additional savings in total computation time. Small-sized instance tests follow similar trends but with a more significant average cost increase (4% for large approximations) and lesser computational improvements (15% for large approximations). These findings imply that approximations have a greater influence on the total solution and lesser benefits for relatively small issue sizes.

5.3.3. *Impact of maximum batch size (Step III)*

Increasing the maximum batch size allows for more efficient usage of resources, but it is at the same limited by SLAs (promised delivery time) and capacity constraints. The results of setting different threshold sizes for batches are illustrated in Fig. 14. As expected, batching two or more orders allows for significant cost reductions. The highest marginal gain (29.2%) occurs when two orders can be batched together rather forcing the algorithm to serve orders individually. This allows for better utilization of the available couriers, whose employment decreases from 100% to 82.1% (with 17.9% of spare supply that could be employed in the following optimization rounds). Interestingly, since the investigated instances reflect real-world demand, supply and time and capacity constraints, further increases of the maximum batch size yield relatively small and quickly diminishing marginal gains in terms of costs (7.8% when increasing the maximum batch size from 2 to 3; 0.0% when increasing the maximum batch size from 3 to 4). This can be explained by SLA and courier capacity constraints preventing the creation of larger batches. Even when allowing for a maximum batch size of 3, only 5% of the created batches reach this threshold in the instances characterized by high order densities. This is an important aspect to consider for real-world implementation. It is also worth mentioning that real-world scenarios, characterized by uncertain travel times and meal preparation times would likely be characterized by even more 'conservative' maximum batch sizes to limit the risk of failing to comply with promised delivery times, which typically increases in batch size.

5.3.4. *Impact of local search (Step IV)*

Step IV performs a series of moves to improve the final OBA solution by identifying batches and assignments missed due to clustering performed in Step I (see Section 4.2.4 for further details). In this phase, only the cheapest potential new batches and corresponding assignments are included in the final auxiliary graph to limit the computational effort. Therefore, only the most

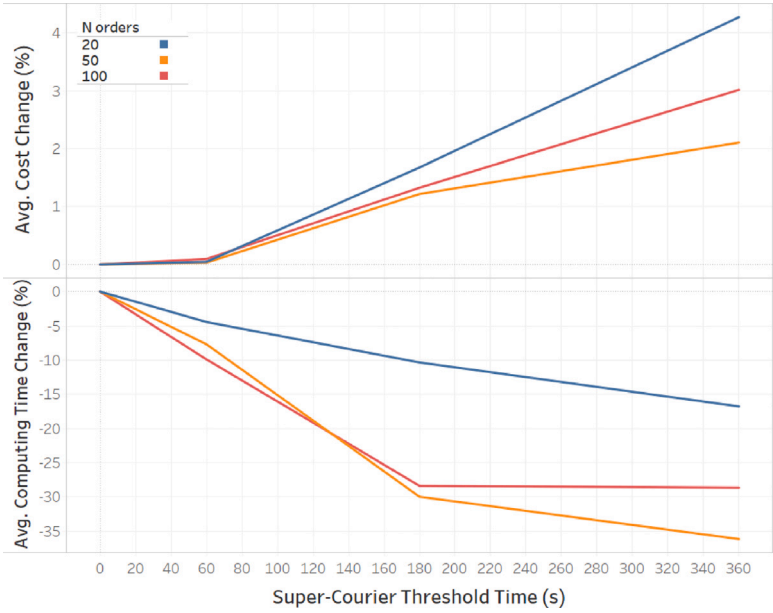


Fig. 13. Super-courier impacts overview.

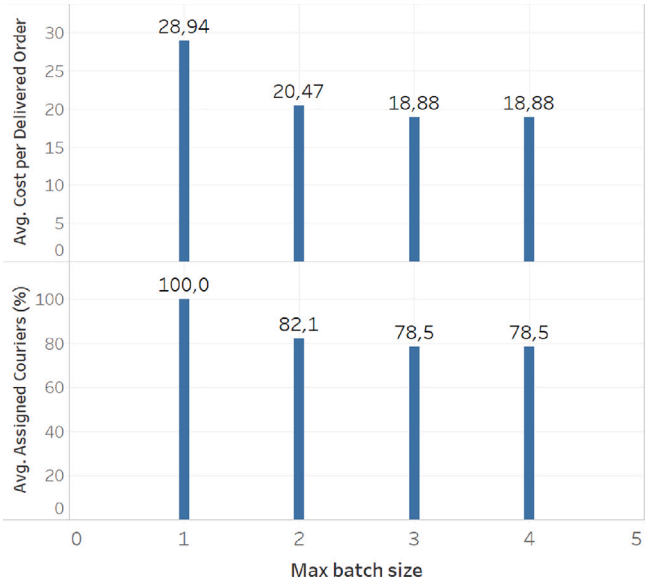


Fig. 14. Effects of different maximum batch sizes.

promising PDP routes (according to the new similarity measure) are considered during the insertion and merging moves based on threshold parameters ψ and ψ'. The effects of different LS parameters (ψ, ψ'), which are varied between 0.1 and 0.2 for different sizes of the OBA problem are illustrated in Fig. 15. As expected, marginal cost reductions are achieved by increasing the LS efforts, with gains between 0.2% and 1.5% depending on the size of the problem. On the other hand, such LS parameters can considerably affect the computation times, particularly in large configurations where a threefold increase occurs when increasing the parameters from 0.1 to 0.2. The benefits gained from a more extensive LS seem to weaken as the number of orders grows. This effect is possibly due to a decrease of the LS algorithm's efficacy for increasing problem sizes.

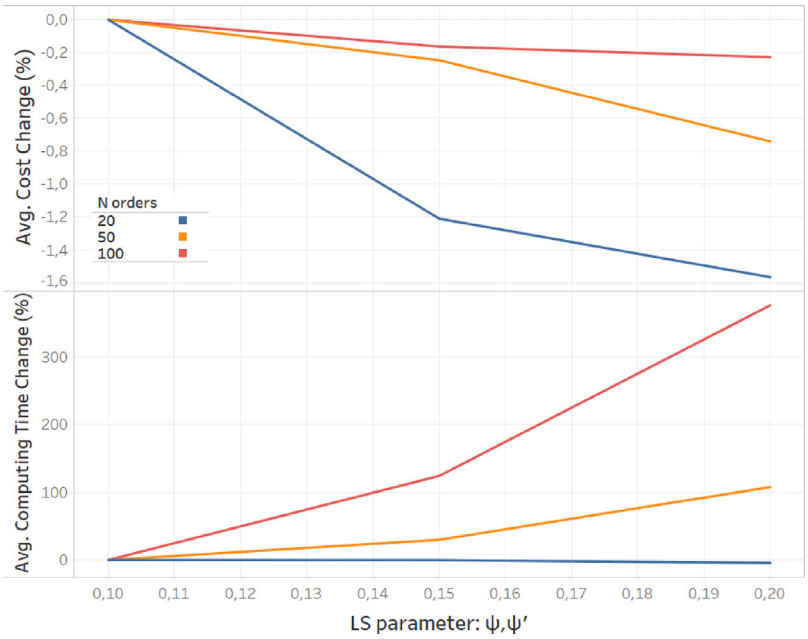
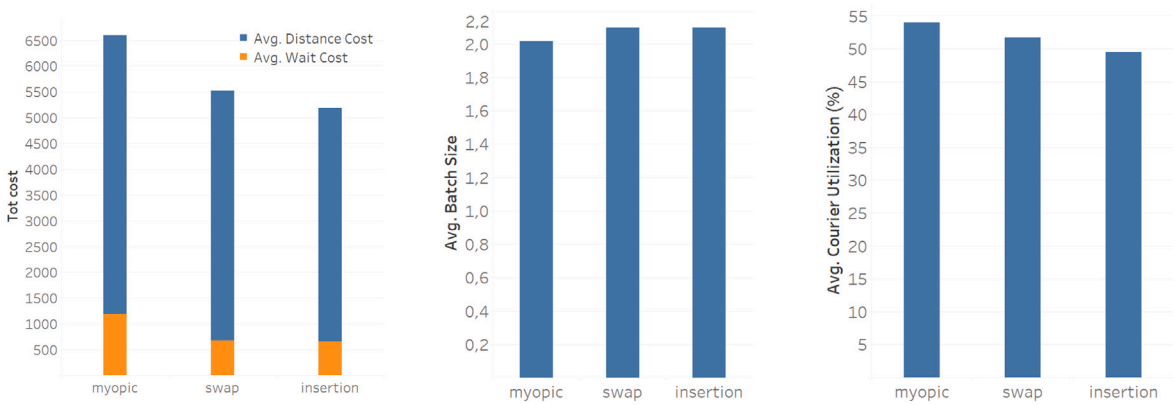


Fig. 15. LS thresholds impacts overview.



(a) Total cost composition for different policies (b) Average batch size for different policies (c) Fleet utilization for different policies

Fig. 16. Performance comparison of Insertion and Swap policy with Myopic policy.

5.4. Policy insights

Due to the possibility of integrating advanced policies throughout the adopted graph-based method, the OBA allows significant performance improvements in dynamic scenarios. Advanced policies, which achieve a higher level of batching, are characterized by lower costs. Significant reductions of costs can be achieved by both the Insertion and Swap policy in relation to the Myopic policy (Fig. 16). Interestingly, the average batch size increases by only 10%, confirming that in real-world instances, feasible and efficient batches are rarely including more than two orders. The Insertion policy seems to more cost-effective than the Swap one as it achieves a larger reduction of 'distance costs' due to the higher level of batching. This result suggests that, while both policies achieve similar batch sizes and courier utilization, the first one can create more efficient routes.

6. Conclusion

While the concept of on-demand food delivery is not new, recent advances in communication and sensing technologies have allowed the development of ODFD platforms relying on non-professional crowd-sourced couriers. Their operations involve solving a dynamic optimization problem, the OBA problem, characterized by multiple theoretical and practical challenges. Orders must be

rapidly assigned to available couriers scattered throughout the network and may be consolidated in batches to minimize delivery costs while avoiding delays. A major prerequisite for addressing this challenge is identifying low-cost, feasible solutions that can be updated in the future optimization epochs following any chosen OBA policy. In this study, we develop an algorithm for the OBA problem.

The OBA problem is first described by highlighting its main features and the identified solution approach. Since the problem is NP-hard (only small instances can be solved exactly), a heuristic is suggested based on clustering and graph theory methods. In the proposed approach, the initial problem is divided through clustering into smaller problems that are tackled independently. For each cluster, a mPDP-TW problem is solved by representing potential batches and assignments as nodes and links of a bipartite graph and solved as an assignment problem. RDP and an approximation based on the concept of super-couriers are used in conjunction to minimize the computations of this procedure. These partial solutions are finally improved by LS, recombination, and integration with dynamic policies. This process is accomplished by leveraging the adopted graph framework for representing solutions. The selection of batches in advanced policies is performed based on soft-tagging, which implies a wait for revealing the assignments.

The heuristic's performance is examined in a variety of scenarios. Two sets of dynamic instances are built with historical data from two ODFD platform's operations. Additionally, a broad range of single-epoch optimization problems (static instances) is investigated to gain a better understanding of the algorithm's properties. For small and large instances (involving several hundreds orders within a two-hour time span), the OBA algorithm achieves high-quality results (with gaps typically below 5 percent) with significant computational time savings. The results are consistent between dynamic and static instances, confirming the capability of the algorithm to provide good quality solutions for different sizes of the problem. The designed heuristic also provides good quality solutions in high-stress conditions with low C2O ratios and high rates of urgent orders due to its capability to prioritize. This phenomenon is well illustrated in the analysis of dynamic instances characterized by significant under-supply. A systematic evaluation of the algorithm's different features (e.g., clustering, approximation to super-couriers, LS moves) highlights trade-offs between accuracy and computational efficiency. The results indicate that maintaining cluster sizes below a certain threshold and aggregating couriers into super-couriers provide considerable computational benefits for relatively low decreases in performance. Increasing the maximum batch size yields little benefit in terms of cost savings for sizes of 3 orders or more due to time and capacity constraints.

Thanks to its flexible structure, the algorithm can straightforwardly integrate advanced policies involving partial re-optimization over multiple optimization steps. Both the proposed *Insertion* and *Swap* policy can achieve significant solution quality improvements compared to the myopic policy due to increased level of batched orders, reduced first-mile cost, and lower couriers' utilization. As expected, such enhancements of the obtained OBA solution come with an increased computational effort. Another strength is the suggested algorithm's ability to parameterize many costs and constraints simultaneously. This property leads to some insightful findings. For example, the marginal benefits of increasing batch sizes are modest for batches of three orders or more since time and capacity constraints become crucial.

There appear several fruitful directions for future research based on the work we present in this paper. Alternative advanced policies, based on the modification of planned batches and assignments over time, could be investigated in future studies. Additionally, other methods incorporating modification and integration of the different algorithm's steps could be investigated. Another interesting direction for future research is a systematic algorithm evaluation in real-world settings characterized by different demand-supply patterns, road networks, and traffic conditions.

Data availability

Data will be made available on request.

Appendix A. Notation

See Table A.2

Appendix B. Problem instances

The problem instances used for this study can be freely accessed at: https://github.com/micheledsimoni/food_platform_instances.

Appendix C. Performance analyses

C.1. Analysis of the postponement mechanism

See Figs. C.17 and C.18 and Tables C.6 and C.7.

Table A.2
Notation table.

Symbol	Definition
t_i	simulation time epoch
h_i	optimization time epoch
o_i	order i belonging to the set of orders O
d_i	courier i belonging to the set of couriers D
t_r	meal preparation time
t_q	promised delivery time
$\hat{k}_i$	last mile cost of order i
$\dot{k}_{ij}$	first mile cost of order j served by courier i
s_{ij}	similarity measure between the pair of orders (i, j)
S	similarity matrix
χ	cluster in configuration of clusters X_i
ϕ_i	silhouette score for a configuration of clusters X_i
T	maximum cluster size
T'	maximum number of clusters
$\dot{k}_{i\chi}$	average first mile cost between courier i and cluster χ
$x_{i\chi}$	decision variable to indicate if courier i serves cluster χ
w_χ	total demand of cluster χ
$u_{i\chi}$	cost of missing orders in cluster χ
ρ	planned batching level parameter
α_χ	cluster-specific parameter for relative urgency of its orders
$G(N, E)$	auxiliary graph
b	courier node in the auxiliary graph
c_{ij}	combined node in the auxiliary graph corresponding to a route serving order i and order j
e_{bc}	links connecting courier nodes to combined nodes in the auxiliary graph
$\kappa_{b,c}$	link weight in the auxiliary graph
$z_{b,c}$	decision variable to indicate if courier node b is assigned to combined node c
y_o	auxiliary variable to express missed order o
u_o	cost of missing order o
p	pickup point belonging to set of pickup points P in the cluster
q	dropoff point belonging to set of dropoff points Q in the cluster
$\sigma(P, Q)$	state representing a sequence of pickup and dropoff nodes
$k(\sigma)$	cumulative cost for state σ
$t(\sigma)$	cumulative time for state σ
d^*	super-courier
t_d	threshold time distance to identify super-couriers
v	threshold number of states per iteration in the RDP
s'_{ij}	new similarity measure between the pair of orders (i, j)
l_{ij}	binary variable to indicate if order i and order j belong to the same cluster
M_o	Set of orders matching with order o
ψ, ψ', ψ''	threshold parameters for LS moves
γ	JIT parameter
I	Inter-interval graph
Z	Set of temporary solutions obtained at the end of step IV

Table B.3
Overview of static instances.

	No. of Orders	Area (km^2)	Courier-to-Order Ratio	Order Density $(1/\mathrm{km}^2)$
Instance21–24	20	8	1.00	2.5
Instance 25–28	20	8	0.50	2.5
Instance 29–32	20	4	1.00	5.0
Instance 33–36	20	4	0.50	5.0
Instance 37–40	30	20	1.00	1.5
Instance 41–44	30	20	0.50	1.5
Instance 45–48	50	20	1.00	2.5
Instance 49–52	50	20	0.50	2.5
Instance 53–56	50	10	1.00	5.0
Instance 57–60	50	10	0.50	5.0
Instance 61–64	100	20	1.00	5.0
Instance 65–68	100	20	0.50	5.0

C.2. Benchmark Grubhub

B2C business-to-consumer

ODFD On-Demand Food Delivery

Table B.4
Input parameters for dynamic instances (Instance 1–20).

Parameter	Value
Simulation period (h)	2 h
Simulation time step (s)	2 s
Optimization time step (s)	2 m
Demand (orders/h)	1,000
Average order size (# items)	2
Area (km2)	10
Mean restaurant-customer distance (km)	2
Mean SLA (m)	25
Mean preparation time (m)	10
Couriers' travel speed (km/h)	20
Couriers' capacity (# items)	6
JIT (min)	5

Table B.5
Overview of high stress instances.

	No. of orders	Area (km^2)	Courier-to-Order Ratio	Order Density $(1/km^2)$	Urgent orders (%)
Instance 69–72	20	4	0.7	5.0	50
Instance 73–76	20	4	0.7	5.0	80
Instance 77–80	20	4	1.0	5.0	50
Instance 81–84	20	4	1.0	5.0	80
Instance 85–88	40	8	0.7	5.0	50
Instance 89–92	40	8	1.0	5.0	80
Instance 93–96	100	20	0.7	5.0	50
Instance 97–100	100	20	0.7	5.0	80

Table C.6
Heuristic performance for high stress instances (Instance 69–84).

Instance	Optimal results				Heuristic results				Gap (%)
	Total costs	Postponed orders %	Cost postponed	Avg. slack postponed	Total costs	Postponed orders %	Cost postponed	Avg. slack postponed	
69	477.9	5	111.2	326	477.9	5	111.2	326	0.0
70	478.5	5	117.3	368	478.5	5	117.3	368	0.0
71	268.1	0	0.0	0	284.6	0	0.0	0	6.2
72	461.2	0	108.8	222	461.2	5	108.8	222	0.0
73	475.0	5	119.6	344	485.7	5	119.6	344	2.3
74	490.8	5	124.5	226	503.2	5	119.8	512	2.5
75	454.9	5	112.6	339	466.3	5	112.6	339	2.5
76	224.6	0	0.0	0	231.0	0	0.0	0	2.8
77	1,958.4	40	903.0	474	1,958.4	40	903.0	474	0.0
78	1,987.8	40	917.0	503	1,995.5	40	918.3	512	0.4
79	1,957.8	40	903.1	418	1,968.8	40	906.6	420	0.6
80	2,006.8	40	923.6	425	2,021.1	40	930.3	422	0.7
81	2,033.3	40	945.8	430	2,060.0	40	952.3	431	1.3
82	2,065.3	40	964.4	483	2,091.0	40	967.0	359	1.2
83	1,890.2	35	872.0	?	2,145.0	40	993.0	374	13.5
84	2,082.3	40	973.1	428	2,082.3	40	973.1	428	0.0

OBA order batching and assignment

VRP vehicle routing problem

PDP pickup and delivery problem

PDPTW pickup and delivery problem with time windows

mPDP-TW multi-vehicle pickup-and-delivery problem with time windows

LS local search

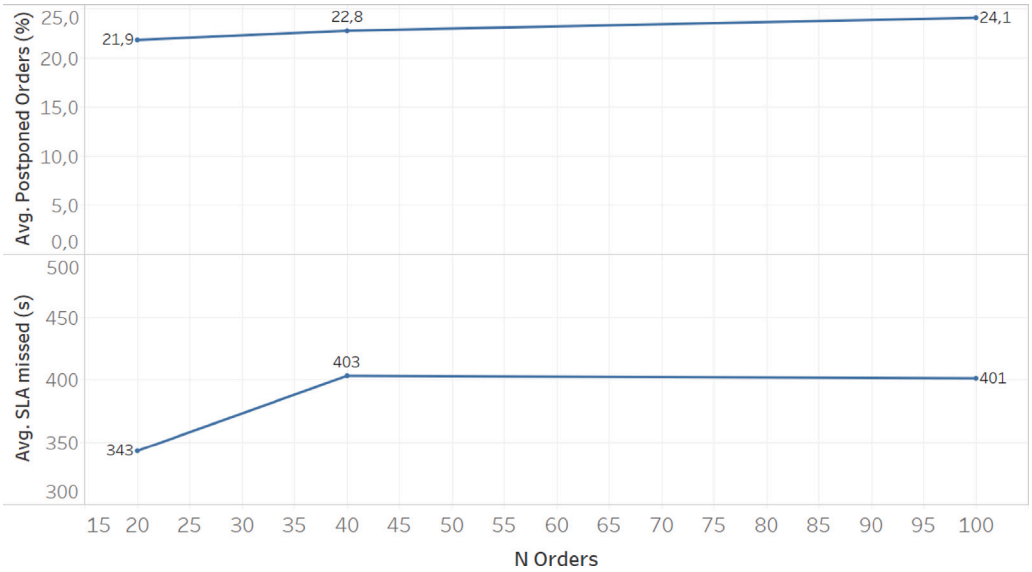


Fig. C.17. Heuristic performance for different levels of high stress orders.

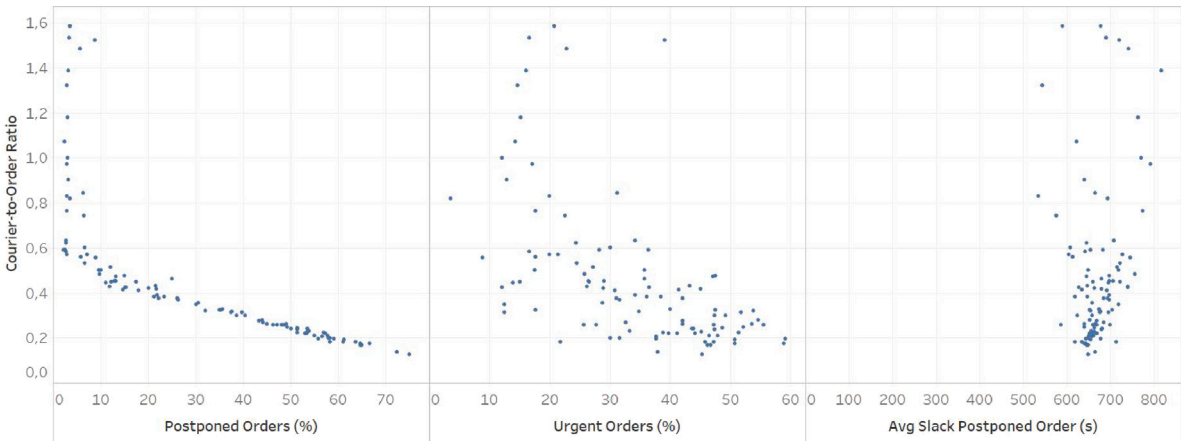


Fig. C.18. Order postponement behavior under stress conditions in dynamic instances.

Table C.7
Performance benchmark of OBA with Grubhub instances.

Instance	Total orders	Avg. batch size	OBA costs	Grubhub costs	Courier occupancy (%)
0o50t75s1p100	252	2.16	724.7	2,490.0	44.3
0o50t75s1p125	252	2.21	727.8	2,460.0	44.0
0o50t75s2p100	252	2.19	781.9	2,480.0	49.6
0o50t75s2p125	252	2.20	799.3	2,480.0	48.1
0r50t100s1p100	242	2.24	1,470.1	2,292.3	54.6
0r50t100s1p125	242	2.22	2,572.6	2,280.1	53.6
0r50t100s2p100	242	2.16	1,254.9	2,359.9	64.2
0r50t100s2p125	242	2.25	1,247.4	2,337.1	62.1
0r50t75s1p100	242	2.19	1,367.0	2,347.3	50.0
0r50t75s1p125	242	2.20	968.9	2,310.0	45.7
0r50t75s2p100	242	2.25	862.1	2,354.5	38.9
0r50t75s2p125	242	2.25	871.1	2,351.7	37.1
0o100t75s1p100	505	2.44	1,868.6	4,923.6	50.4
0o100t75s1p125	505	2.45	1,994.4	4,933.6	48.3
0o50t100s1p100	252	2.30	976.0	2,524.5	56.3
0o50t100s1p125	252	2.24	1,087.7	2,491.3	49.3

DIANA Divisive Analysis

MIP mixed-integer programming

ILP integer linear program

RDP restricted dynamic programming

SLA Service Level Agreement

JIT just-in-time

C2O courier-to-order

References

Agatz, N., Erera, A.L., Savelsbergh, M.W., Wang, X., 2011. Dynamic ride-sharing: A simulation study in metro Atlanta. Procedia Soc. Behav. Sci. 17, 532–550.

Alonso-Mora, J., Samaranayake, S., Wallar, A., Frazzoli, E., Rus, D., 2017. On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment. Proc. Natl. Acad. Sci. 114 (3), 462–467.

Arslan, A.M., Agatz, N., Kroon, L., Zuidwijk, R., 2019. Crowdsourced delivery—A dynamic pickup and delivery problem with ad hoc drivers. Transp. Sci. 53 (1), 222–235.

Baldacci, R., Bartolini, E., Mingozzi, A., 2011. An exact algorithm for the pickup and delivery problem with time windows. Oper. Res. 59 (2), 414–426.

Battarra, M., Cordeau, J., Iori, M., 2014. Chapter 6: pickup-and-delivery problems for goods transportation. In: Vehicle Routing: Problems, Methods, and Applications, second ed. Society for Industrial and Applied Mathematics, pp. 161–191.

Bent, R., Van Hentenryck, P., 2006. A two-stage hybrid algorithm for pickup and delivery vehicle routing problems with time windows. Comput. Oper. Res. 33 (4), 875–893.

Berbeglia, G., Cordeau, J.F., Laporte, G., 2010. Dynamic pickup and delivery problems. European J. Oper. Res. 202 (1), 8–15. http://dx.doi.org/10.1016/j.ejor.2009.04.024.

Bertsimas, D., Jaillet, P., Martin, S., 2019. Online vehicle routing: The edge of optimization in large-scale applications. Oper. Res. 67 (1), 143–162.

Bodin, Sexton, 1986. The multi-vehicle subscriber dial-a-ride problem. TIMS Studies Manage. Science 26, 73–86.

Bongiovanni, C., Kaspi, M., Geroliminis, N., 2019. The electric autonomous dial-a-ride problem. Transp. Res. B 122, 436–456.

Braekers, K., Caris, A., Janssens, G., 2014. Exact and meta-heuristic approach for a general heterogeneous dial-a-ride problem with multiple depots. Transp. Res. B 67, 166–186.

Braekers, K., Ramaekers, K., Van Nieuwenhuyse, I., 2016. The vehicle routing problem: State of the art classification and review. Comput. Ind. Eng. 99, 300–313.

Chen, X.M., Zheng, H., Ke, J., Yang, H., 2020. Dynamic optimization strategies for on-demand ride services platform: Surge pricing, commission rate, and incentives. Transp. Res. B 138, 23–45.

Coppola, D., 2022. Online Food Delivery - Statistics & Facts. Statista.

Cordeau, J., Laporte, G., 2003. A tabu search heuristic for the static multi-vehicle dial-a-ride problem. Transp. Res. B 37 (6), 579–594.

Cordeau, J.-F., Laporte, G., 2007. The dial-a-ride problem: models and algorithms. Ann. Oper. Res. 153 (1), 29–46.

Cordeau, J.-F., Laporte, G., Ropke, S., 2008. Recent models and algorithms for one-to-one pickup and delivery problems. In: Golden, B., Raghavan, S., Wasil, E. (Eds.), The Vehicle Routing Problem: Latest Advances and New Challenges. Springer US, Boston, MA, pp. 327–357, URL: http://link.springer.com/10.1007/978-0-387-77778-8_15.

Desrosiers, J., Dumas, Y., Soumis, F., 1986. A dynamic programming solution of the large-scale single-vehicle dial-a-ride problem with time windows". Amer. J. Math. Management Sci. 6, 301–325.

Desrosiers, J., Dumas, Y., Soumis, F., 1988. The multiple vehicle dial-a-ride problem. In: Computer-Aided Transit Scheduling. Springer, Berlin, Heidelberg, pp. 15–27.

Ding, C., He, X., 2002. Cluster merging and splitting in hierarchical clustering algorithms. In: 2002 IEEE International Conference on Data Mining, 2002. Proceedings. IEEE, pp. 139–146.

Dumas, Y., Desrosiers, J., Soumis, F., 1991. The pickup and delivery problem with time windows. European J. Oper. Res. 54 (1), 7–22.

Häme, L., Hakula, H., 2015. A maximum cluster algorithm for checking the feasibility of dial-a-ride instances. Transp. Sci. 49 (2), 295–310.

Hess, A., Spinler, S., Winkenbach, M., 2021. Real-time demand forecasting for an urban delivery platform. Transp. Res. E Logist. Transp. Rev. 145, 102147.

Hirschberg, C., Rajko, A., Schumacher, T., Wrulich, M., 2016. The Changing Market for Food Delivery. McKinsey & Company.

Ho, S.C., Szeto, W.Y., Kuo, Y.-H., Leung, J.M., Petering, M., Tou, T.W., 2018. A survey of dial-a-ride problems: Literature review and recent developments. Transp. Res. B 111, 395–421.

Hyland, M., Mahmassani, H.S., 2018. Dynamic autonomous vehicle fleet operations: Optimization-based strategies to assign AVs to immediate traveler demand requests. Transp. Res. C 92, 278–297.

Lahyani, R., Khemakhem, M., Semet, F., 2015. Rich vehicle routing problems: From a taxonomy to a definition. European J. Oper. Res. 241 (1), 1–14. http://dx.doi.org/10.1016/j.ejor.2014.07.048.

Le, T.V., Stathopoulos, A., Van Woensel, T., Ukkusuri, S.V., 2019. Supply, demand, operations, and management of crowd-shipping services: A review and empirical evidence. Transp. Res. C 103, 83–103.

Le, T.V., Ukkusuri, S.V., Xue, J., Van Woensel, T., 2021. Designing pricing and compensation schemes by integrating matching and routing models for crowd-shipping systems. Transp. Res. E Logist. Transp. Rev. 149, 102209.

Liang, X., de Almeida Correia, G.H., An, K., van Arem, B., 2020. Automated taxis' dial-a-ride problem with ride-sharing considering congestion-based dynamic travel times. Transp. Res. C 112, 260–281.

Liu, Y., Li, Z., Xiong, H., Gao, X., Wu, J., 2010. Understanding of internal clustering validation measures. In: 2010 IEEE International Conference on Data Mining. IEEE, pp. 911–916.

Ma, T.-Y., Rasulkhani, S., Chow, J.Y., Klein, S., 2019. A dynamic ridesharing dispatch and idle vehicle repositioning strategy with integrated transit transfers. Transp. Res. E Logist. Transp. Rev. 128, 417–442.

Macrina, G., Pugliese, L.D.P., Guerriero, F., Laporte, G., 2020. Crowd-shipping with time windows and transshipment nodes. Comput. Oper. Res. 113, 104806.

Mahmoudi, M., Chen, J., Shi, T., Zhang, Y., Zhou, X., 2019. A cumulative service state representation for the pickup and delivery problem with transfers. Transp. Res. B 129, 351–380.

Mahmoudi, M., Zhou, X., 2016. Finding optimal solutions for vehicle routing problem with pickup and delivery services with time windows: A dynamic programming approach based on state–space–time network representations. Transp. Res. B 89, 19–42.
Masoud, N., Jayakrishnan, R., 2017. A real-time algorithm to solve the peer-to-peer ride-matching problem in a flexible ridesharing system. Transp. Res. B 106, 218–236.
Mitrović-Minić, S., Laporte, G., 2004. Waiting strategies for the dynamic pickup and delivery problem with time windows. Transp. Res. B 38 (7), 635–655.
Mourad, A., Puchinger, J., Chu, C., 2019. A survey of models and algorithms for optimizing shared mobility. Transp. Res. B 123, 323–346.
Pankratz, G., 2005. A grouping genetic algorithm for the pickup and delivery problem with time windows. Or Spectrum. 27 (1), 21–41.
Parragh, S., Doerner, K., Hartl, R., 2010. Variable neighborhood search for the dial-a-ride problem. Comput. Oper. Res. 37 (6), 1129–1138.
Qian, X., Zhang, W., Ukkusuri, S.V., Yang, C., 2017. Optimal assignment and incentive design in the taxi group ride problem. Transp. Res. B 103, 208–226.
Rasulkhani, S., Chow, J.Y., 2019. Route-cost-assignment with joint user and operator behavior as a many-to-one stable matching assignment game. Transp. Res. B 124, 60–81.
Reyes, D., Erera, A., Savelsbergh, M., Sahasrabudhe, S., O'Neil, R., 2018. The meal delivery routing problem. Optimization Online.
Ritzinger, U., Puchinger, J., Hartl, R., 2016. Dynamic programming based metaheuristics for the dial-a-ride problem. Ann. Oper. Res. 236 (2), 341–358.
Rokach, L., Maimon, O., 2005. Clustering methods. In: Data Mining and Knowledge Discovery Handbook. Springer, pp. 321–352.
Ropke, S., Cordeau, J., 2009. Branch and cut and price for the pickup and delivery problem with time windows. Transp. Sci. 43 (3), 267–286.
Ropke, S., Pisinger, D., 2006. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transp. Sci. 40 (4), 455–472.
Rousseeuw, P.J., Kaufman, L., 1990. Finding Groups in Data. Wiley Online Library, Hoboken.
Santi, P., Resta, G., Szell, M., Sobolevsky, S., Strogatz, S., Ratti, C., 2014. Quantifying the benefits of vehicle pooling with shareability networks. Proc. Natl. Acad. Sci. 111 (37), 13290–13294.
Santini, A., Schneider, M., Vidal, T., Vigo, D., 2021. Decomposition Strategies for Vehicle Routing Heuristics. Working Paper, URL: http://www.optimization-online.org/DB_FILE/2021/03/8316.pdf.
Savelsbergh, M.W.P., Sol, M., 1995. The general pickup and delivery problem. Transp. Sci. 29 (1), 17–29. http://dx.doi.org/10.1287/trsc.29.1.17.
Seghezzi, A., Winkenbach, M., Mangiaracina, R., 2021. On-demand food delivery: a systematic literature review. Int. J. Logist. Manag.
Simonetto, A., Monteil, J., Gambella, C., 2019. Real-time city-scale ridesharing via linear assignment problems. Transp. Res. C 101, 208–232.
Steever, Z., Karwan, M., Murray, C., 2019. Dynamic courier routing for a food delivery service. Comput. Oper. Res. 107, 173–188.
Stiglic, M., Agatz, N., Savelsbergh, M., Gradisar, M., 2016. Making dynamic ride-sharing work: The impact of driver and rider flexibility. Transp. Res. E Logist. Transp. Rev. 91, 190–207.
Sun, P., Veelenturf, L., Hewitt, M., Van Woensel, T., 2018. The time-dependent pickup and delivery problem with time windows. Transp. Res. B 116, 1–24.
Vakulenko, Y., Shams, P., Hellström, D., Hjort, K., 2019. Service innovation in e-commerce last mile delivery: Mapping the e-customer journey. J. Bus. Res. 101, 461–468.
Vazifeh, M.M., Santi, P., Resta, G., Strogatz, S.H., Ratti, C., 2018. Addressing the minimum fleet problem in on-demand urban mobility. Nature 557 (7706), 534–538.
Vincent, F.Y., Jodiawan, P., Redi, A.P., 2022. Crowd-shipping problem with time windows, transshipment nodes, and delivery options. Transp. Res. E Logist. Transp. Rev. 157, 102545.
Wang, H., Yang, H., 2019. Ridesourcing systems: A framework and review. Transp. Res. B 129, 122–155.
Xu, H., Chen, Z., Rajagopal, S., Arunapuram, S., 2003. Solving a practical pickup and delivery problem. Transp. Sci. 37 (3), 347–364.
Yildiz, B., Savelsbergh, M., 2019. Provably high-quality solutions for the meal delivery routing problem. Transp. Sci. 53 (5), 1372–1388.